



MONASH University

FIT3164 Data Science Project 2

User Guide and Testing Report

Datalake for the Monash Malaysia Microbiome Research Centre

Group Name: **MDS03**

Team Members:

Chang Lin Ong	(32024509)
Jun Ho Chew	(32409966)
Jia Jing Chai	(31920772)
Tharani Prathaban	(32886047)

Team supervised by:

A/Prof Wong Kok Sheik

Table of content

Section 1: End user guide.....	3
1.1 Introduction.....	3
1.2 Accessing the Platform.....	3
1.2.1 Upload Interface (Streamlit App).....	3
1.2.2 Athena Console.....	3
1.3 Uploading Data via Streamlit.....	3
1.3.1 Required Files.....	3
1.3.2 Validation Features.....	4
1.3.3 Review & Fix Validation Messages.....	4
1.3.4 Upload to S3.....	6
1.4 Querying Data Using Athena.....	9
1.4.0 SQL Guide (Provided Separately).....	9
1.4.1 Getting Started.....	9
1.4.2 Example Join Query.....	9
1.5 Known Limitations.....	10
1.6 Exiting the Platform.....	10
Section 2: Technical guide.....	11
2.1 Introduction.....	11
2.2 Local Deployment.....	11
2.2.1 Prerequisites.....	11
2.2.2 Setup Instructions.....	11
2.3 Cloud Access (Streamlit Hosted App).....	12
2.3.1 How to Access the App.....	12
Section 3: Software Test Introduction & Test Plan.....	14
3.1 Introduction.....	14
3.2 Test Objectives.....	14
3.3 Test Approach.....	14
Section 4: Black Box Testing.....	15
4.1 Front End Testing.....	15
4.2 AWS Cloud Testing (BackEnd).....	31
Section 5: Integration Test.....	41
Section 6: End User Testing.....	49
Section 7: Recommendations for Improvement.....	66
7.1 Enhancement to front-end validation for CSV files.....	66
7.2 Optimization of processing speed for the web interface.....	66
7.3 Improve the clarity of error messages.....	66
Section 8: Test Limitations.....	67
8.1 Limited testing with diverse, dynamically structured real-world datasets.....	67
8.2 Current error messages may not be sufficiently clear.....	67
8.3 Unperformed Tests.....	68

Section 1: End user guide

1.1 Introduction

This end user guide is intended for researchers and staff at the Monash Malaysia Microbiome Research Centre (MMRC) who use the Data Lake platform to upload and query microbiome-related datasets. It provides step-by-step instructions on how to validate and upload CSV files via the Streamlit web interface, and how to perform data queries using Amazon Athena.

1.2 Accessing the Platform

1.2.1 Upload Interface (Streamlit App)

- Navigate to: <https://mmrc-datalake-data-upload.streamlit.app/>

1.2.2 Athena Console

- Visit: <https://console.aws.amazon.com/athena/>
- Select region: ap-southeast-1
- Open the **Query Editor**

1.3 Uploading Data via Streamlit

1.3.1 Required Files

Users must upload the following five CSV files:

- bacterial-abundance.csv
- bacterial-pathway.csv
- enzymes-abundance.csv
- nmr-asics-abundance.csv
- participant-metadata.csv
- Optional: taxonomy.csv

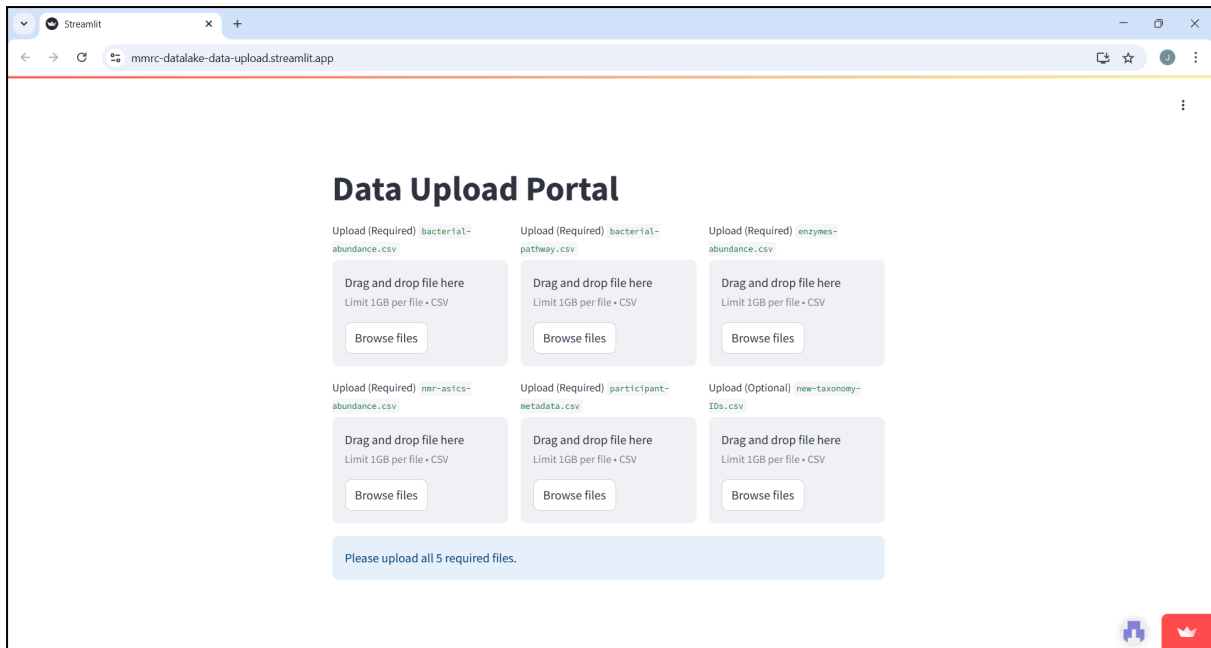


Figure 1: Screenshot of the data upload page in Streamlit

1.3.2 Validation Features

The app automatically performs:

- Intra-file duplicate detection (within each file)
- Cross-file sample ID consistency checks
- Duplicate sample ID check against DynamoDB (for participant metadata)

1.3.3 Review & Fix Validation Messages

If any issues are detected:

- A list of errors will be displayed on the interface
- The app will provide interactive buttons to resolve them:
 - For duplicate sample IDs, we will retain the first occurrence and discard the rest
 - For missing IDs: remove rows with missing sample_id
- Once resolved, the files are re-validated automatically

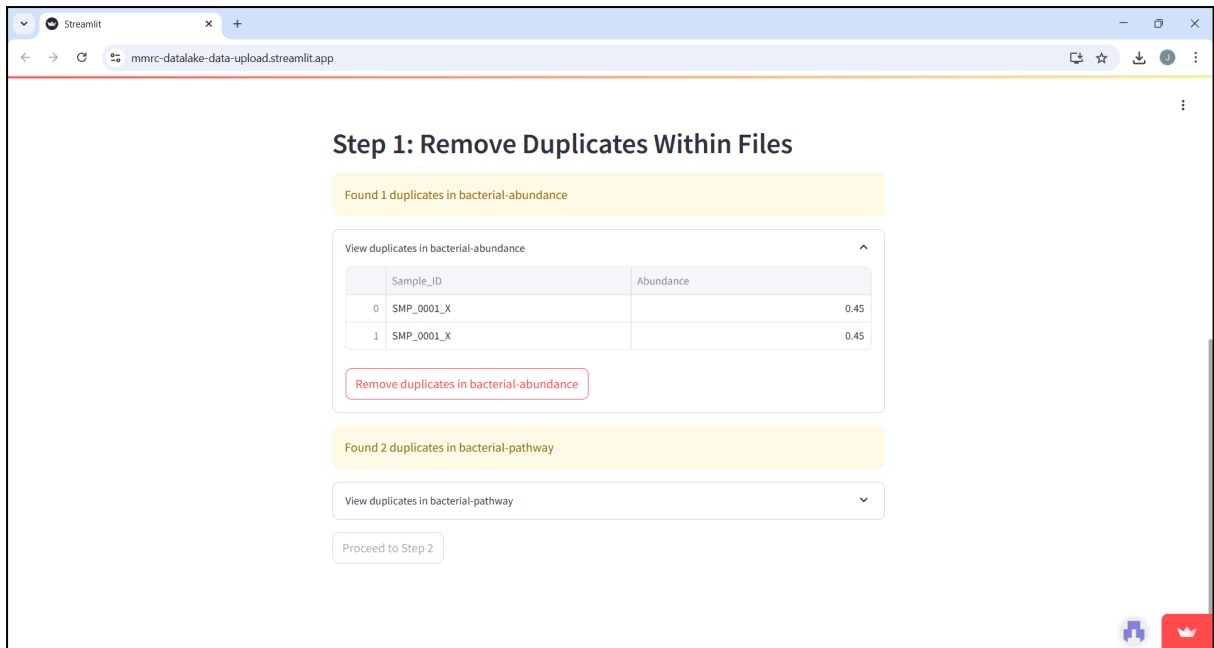


Figure 2: The app first scans and detects any duplicate files, then provides an option to remove the duplicate records

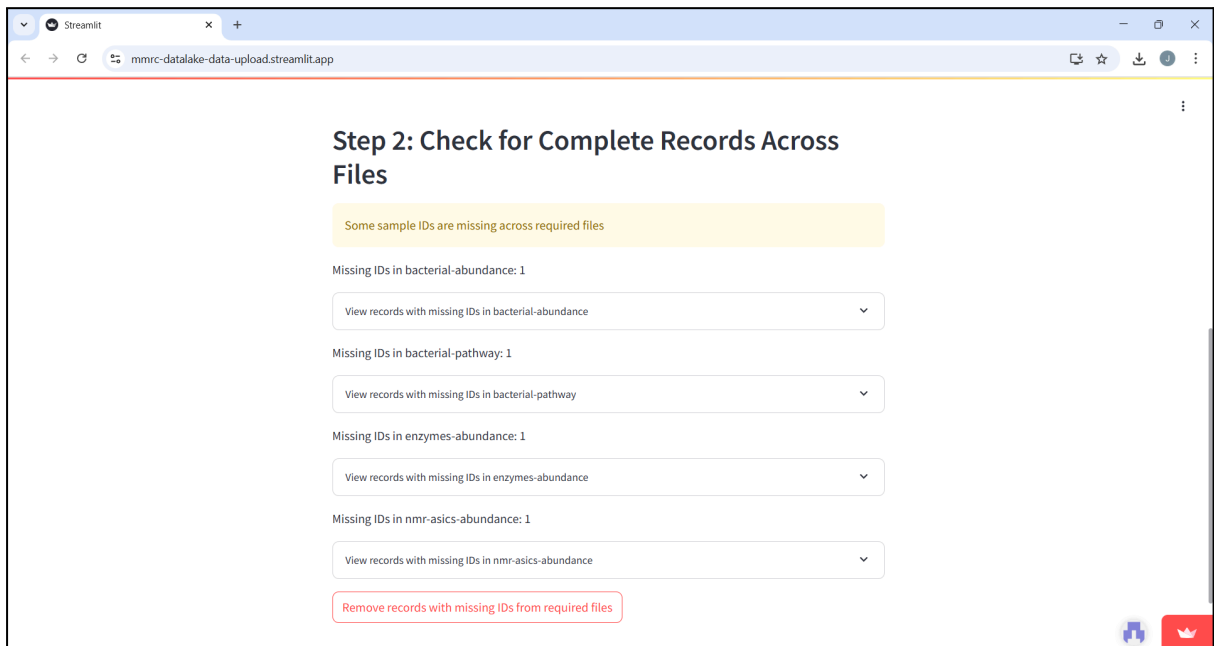


Figure 3: The app then checks for complete records across files, reports any missing IDs, and provides an option to remove those records

1.3.4 Upload to S3

Once validation passes:

- Click the “**Upload to S3**” button
- Files are uploaded to their respective raw/ folders in AWS S3
- Sample ID hashes are updated in DynamoDB for duplicate tracking

After upload:

- An AWS Glue job will automatically start to process the uploaded data and move it to the staging/ layer
- You will receive an email notification via AWS SNS confirming that the Glue job has started

Upon completion or failure of the Glue job:

- Another email will be sent notifying you of the job’s success or failure status, along with relevant details

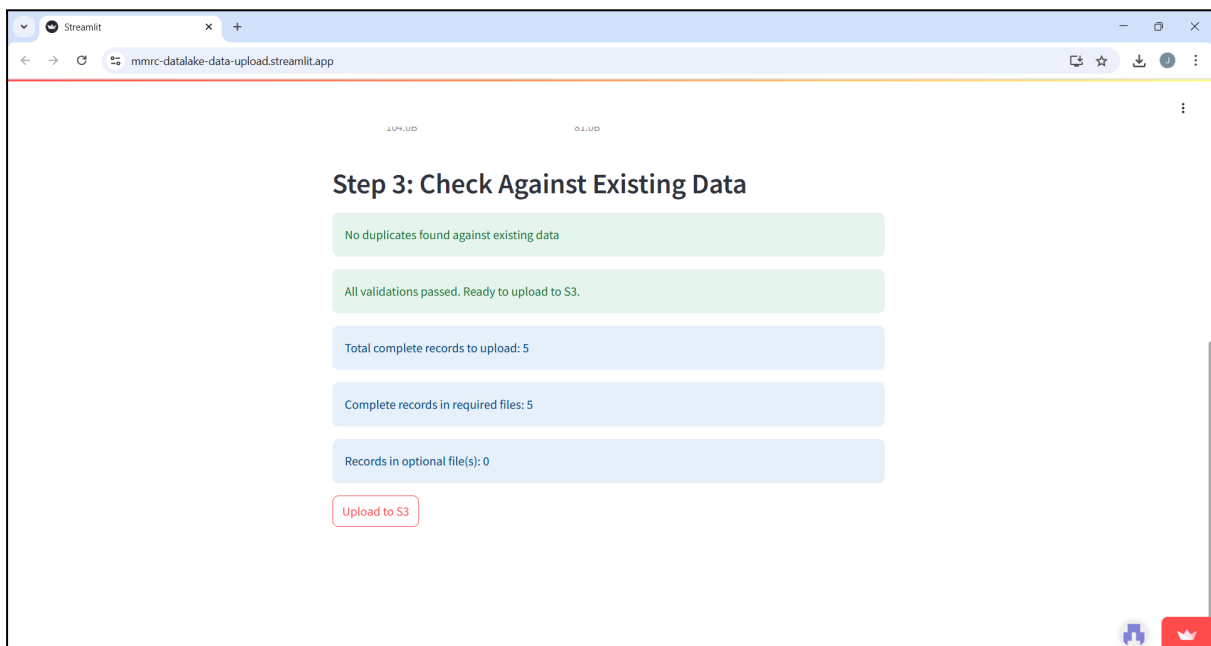


Figure 4: The app checks the data against existing records. If all validations pass, the data is ready and can be uploaded to S3 using the provided button

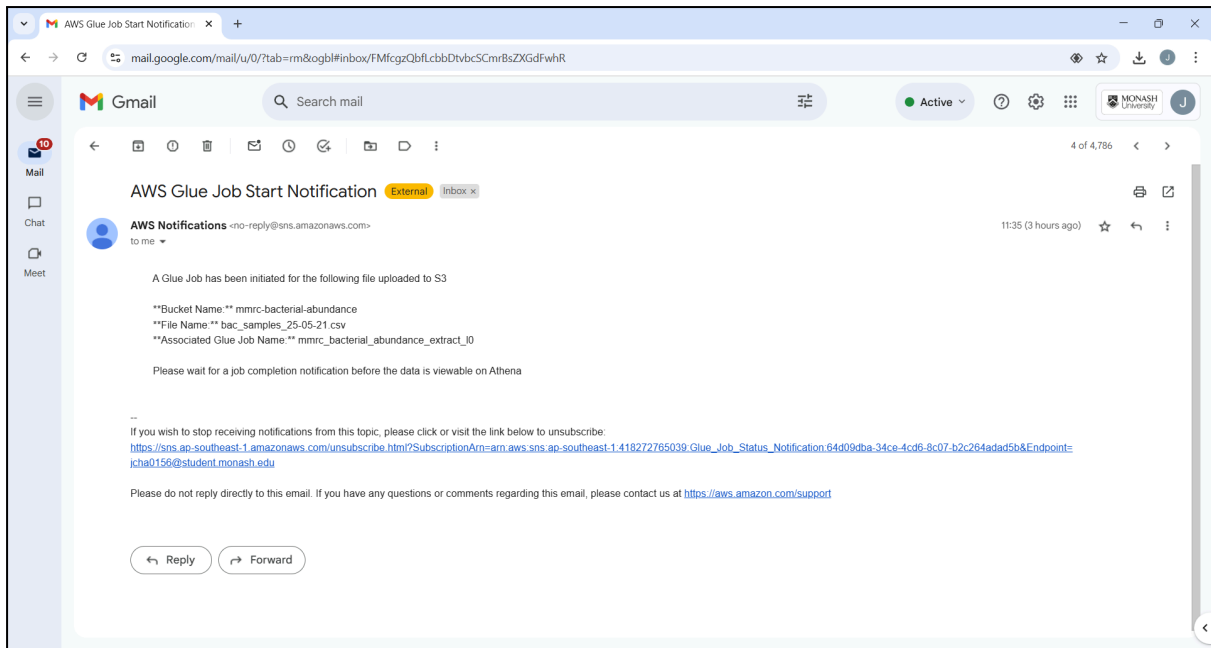


Figure 5: A simple SNS notification is sent via email to inform that the Glue job has started

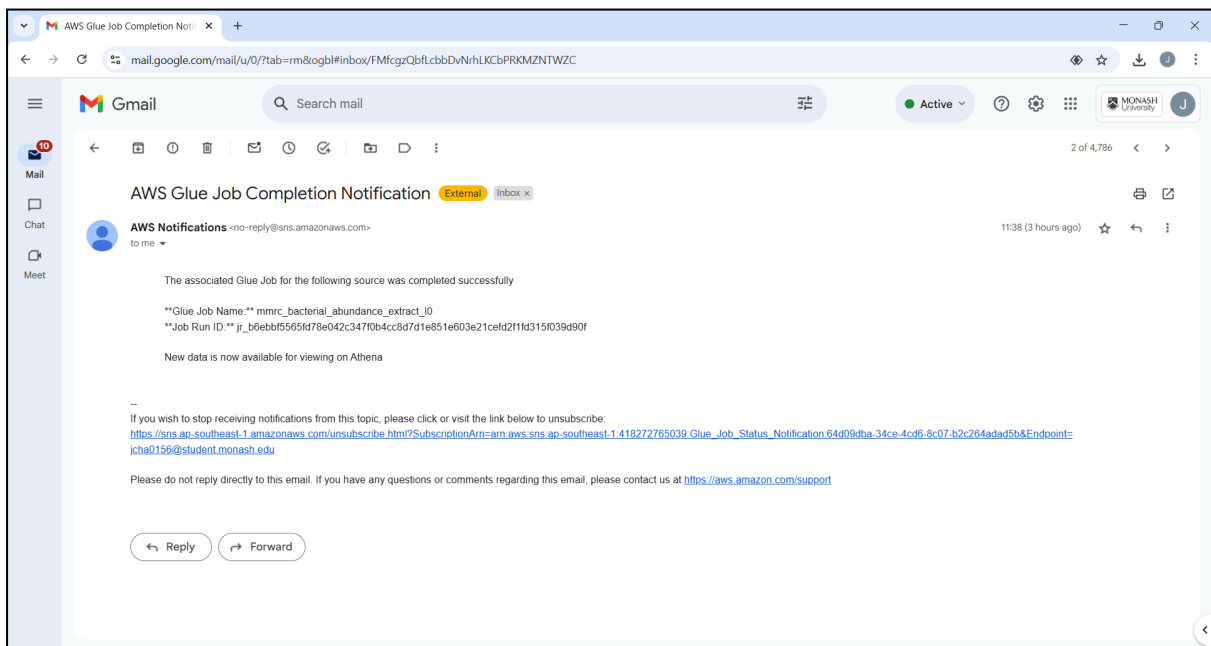


Figure 6: An SNS notification via email informs the user that the Glue job has completed successfully and the data is ready to be queried through Athena

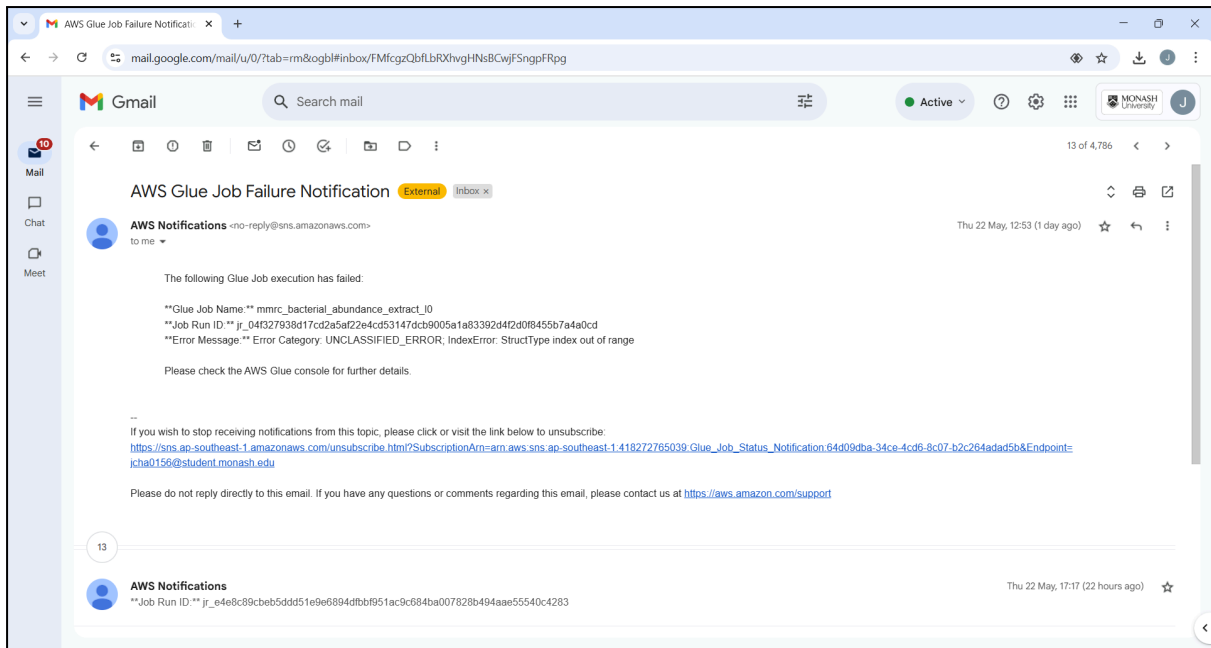


Figure 7: An SNS notification informs the user that the Glue job has failed and provides an error message with details about the issue

1.4 Querying Data Using Athena

1.4.0 SQL Guide (Provided Separately)

A separate SQL user guide document will be made available to end users. It contains step-by-step instructions and example queries to help researchers perform tasks such as filtering by age, joining tables across datasets, and exploring microbial or metabolite data.

This guide is intended to support users who may not be familiar with SQL or the Athena query environment, ensuring they can access and analyze uploaded data effectively.

1.4.1 Getting Started

- Select database: mmrc_microbiome
- Sample queries:

```
-- Show all tables
SHOW TABLES;

-- Describe a table
DESCRIBE mmrc_microbiome.mmrc_participant_metadata;

-- Preview data
SELECT * FROM "mmrc_microbiome"."mmrc_bacterial_abundance_sam"
limit 10;
```

1.4.2 Example Join Query

```
SELECT *
FROM "mmrc_microbiome"."mmrc_participant_metadata" a
JOIN "mmrc_microbiome"."mmrc_bacterial_abundance_sam" b
ON a.sample_id = b.sample_id;
```

Query results

Query status

✔ Completed

Time in queue: 59 ms

Run time: 2.561 sec

Data scanned: 80.97 KB

Results (99)

📄 Copy

Download results CSV

🔍 Search rows

< 1 > ⚙

# ▾	sample_id ▾	source_file ▾	batch_year ▾	batch_month ▾	batch_day ▾	by ▾	agegroup ▾	sago ▾
1	sample5001	Metadata_table.csv	2025	05	05	1954	E	B
2	sample5002	Metadata_table.csv	2025	05	05	1958	E	B
3	sample5003	Metadata_table.csv	2025	05	05	1956	E	A
4	sample5004	Metadata_table.csv	2025	05	05	1950	F	A
5	sample5005	Metadata_table.csv	2025	05	05	1953	E	A
6	sample5006	Metadata_table.csv	2025	05	05	1965	D	A
7	sample5007	Metadata_table.csv	2025	05	05	1956	E	A
8	sample5008	Metadata_table.csv	2025	05	05	1953	E	B
9	sample5009	Metadata_table.csv	2025	05	05	1957	E	A
10	sample5010	Metadata_table.csv	2025	05	05	1959	E	A

Figure 8: Output of the sample join query result

Results are saved automatically to your Athena S3 results bucket.

1.5 Known Limitations

- Only .csv format is accepted
- Files must match required file names exactly
- Only raw/ folder uploads are supported
- No direct editing of S3 files once uploaded

1.6 Exiting the Platform

- To exit the **Streamlit** app: close the browser tab
- To exit **Athena**: close the browser tab or sign out of the AWS Console

Section 2: Technical guide

2.1 Introduction

This section provides setup and deployment instructions for the Microbiome Data Uploader & Validator web application. The app can be accessed in two ways:

- **Locally:** Run directly on a researcher's or administrator's machine
- **Cloud (Streamlit Cloud):** Hosted remotely and accessed via browser login

Each method is outlined below.

2.2 Local Deployment

2.2.1 Prerequisites

- Python 3.8+
- pip or conda
- AWS IAM credentials with access to:
 - S3 (PutObject, GetObject)
 - DynamoDB (PutItem, GetItem)

2.2.2 Setup Instructions

Step 1: Clone the Repository

```
git clone
https://git.infotech.monash.edu/fit3164-monashmalaysiadatalake/dat
auploadui.git
```

Step 2: Install Dependencies

```
pip install -r requirements.txt
```

Step 3: Set AWS Credentials

Set environment variables in a .env file

For mac user:

```
export AWS_ACCESS_KEY_ID=your-access-key
export AWS_SECRET_ACCESS_KEY=your-secret-key
export AWS_DEFAULT_REGION=ap-southeast-1
```

For windows / linux user:

```
$Env:AWS_ACCESS_KEY_ID      = "your-access-key"
$Env:AWS_SECRET_ACCESS_KEY  = "your-secret-key"
$Env:AWS_DEFAULT_REGION     = "ap-southeast-1"
```

Step 4: Launch the App

For mac user:

```
streamlit run app.py
```

For windows / linux user:

```
python -m streamlit run app.py
```

You will see a local access link like:

```
http://localhost:8501
```

2.3 Cloud Access (Streamlit Hosted App)

The Microbiome Data Uploader & Validator is already deployed on Streamlit Cloud and accessible through a private link.

2.3.1 How to Access the App

1. Go to the following URL: <https://mmrc-datalake-data-upload.streamlit.app/>
2. Log in using your **Google** or **GitHub** account.
You must use the same email that was added to the access list by the administrator.
3. Once logged in, you'll be directed to the uploader interface.

4. From there, you can upload and validate your required CSV files following the instructions in **Section 1.3**.

Note: If you are not granted access, you will see a restricted access message. Contact the system administrator to request access.

Section 3: Software Test Introduction & Test Plan

3.1 Introduction

This test report documents the user testing conducted for the Microbiome Data Uploader & Validator, a Streamlit-based web application developed as part of the MMRC Datalake project. The primary objective of the app is to enable research staff at the Monash Malaysia Microbiome Research Centre (MMRC) to easily validate and upload microbiome-related CSV datasets to AWS S3.

The focus of this testing is on functional correctness, usability, and user experience from the perspective of non-technical users. Testing ensures that the application behaves as expected when accessed through the Streamlit interface and that users can perform key operations independently and accurately.

3.2 Test Objectives

- To verify that users can successfully access the app via Streamlit
- To confirm that all validation checks (duplicate IDs, missing data, DynamoDB conflicts) work correctly
- To validate the upload of corrected datasets to the appropriate S3 buckets
- To assess overall usability and user confidence

3.3 Test Approach

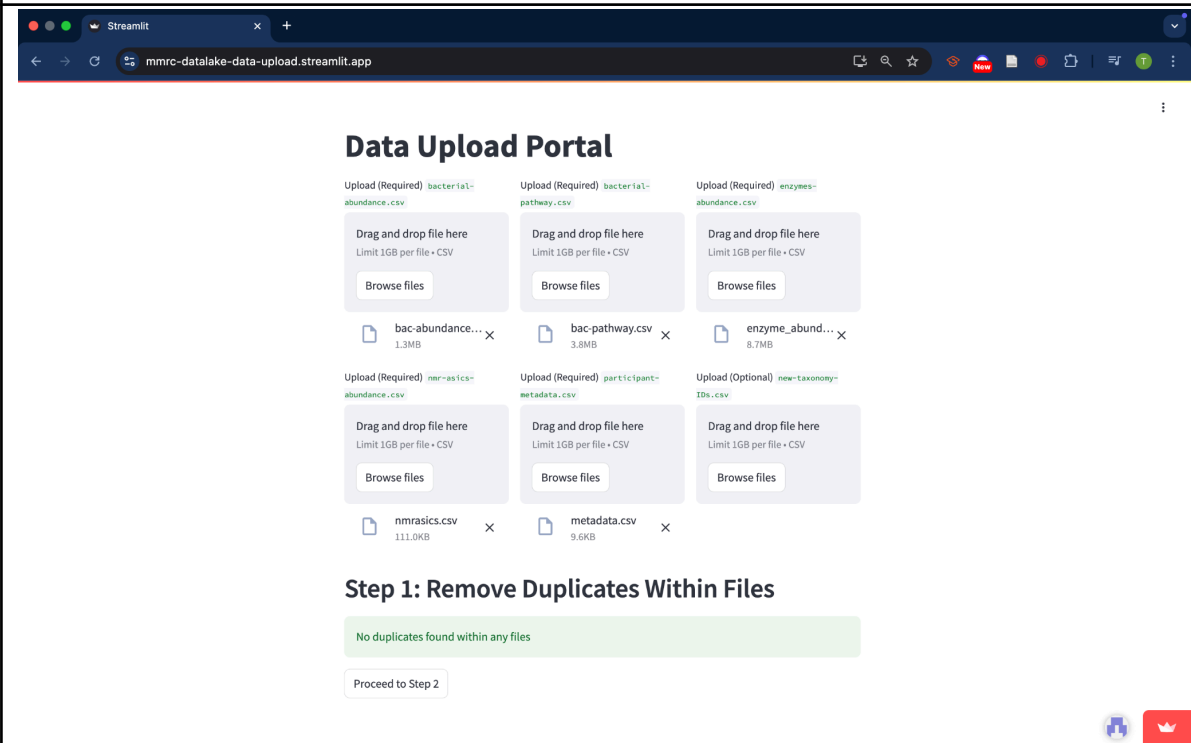
Testing was conducted using **manual, scenario-based walkthroughs** that simulate common user actions within the Streamlit web interface. This approach allows for evaluating both functionality and usability in conditions that closely reflect real-world usage by researchers.

Manual testing was employed not only for **usability and performance evaluation**, but also for verifying individual components (unit testing) and the interactions between them (integration testing). This included testing the validation logic, file handling processes, and backend integrations with AWS S3 and DynamoDB.

By executing step-by-step actions such as uploading files, reviewing validation messages, and resolving errors, testers were able to identify usability gaps and ensure that the system performs reliably across the entire data upload flow.

Section 4: Black Box Testing

4.1 Front End Testing

Test ID	F01
What is being tested?	File upload validation for required files
How is it tested?	Upload all 5 required CSV files
Input	bacterial-abundance.csv, bacterial-pathway.csv, enzymes-abundance.csv, nmr-asics-abundance.csv, participant-metadata.csv
Expected Output	All required files are uploaded, session moves to Step 1 of the validation
Actual Output	Same as expected output
Output Evidence	
 The screenshot displays the 'Data Upload Portal' interface. It features six upload slots arranged in two rows of three. Each slot includes a 'Browse files' button and a file icon. The first five slots are labeled 'Upload (Required)' and correspond to the files listed in the test input: bacterial-abundance.csv (1.3MB), bacterial-pathway.csv (3.8MB), enzymes-abundance.csv (8.7MB), nmr-asics-abundance.csv (111.0KB), and participant-metadata.csv (9.6KB). The sixth slot is labeled 'Upload (Optional)' and corresponds to new-taxonomy-IDs.csv. All required files are shown as uploaded. Below the upload slots, a message states 'No duplicates found within any files' and a button 'Proceed to Step 2' is visible.	
Result	PASS

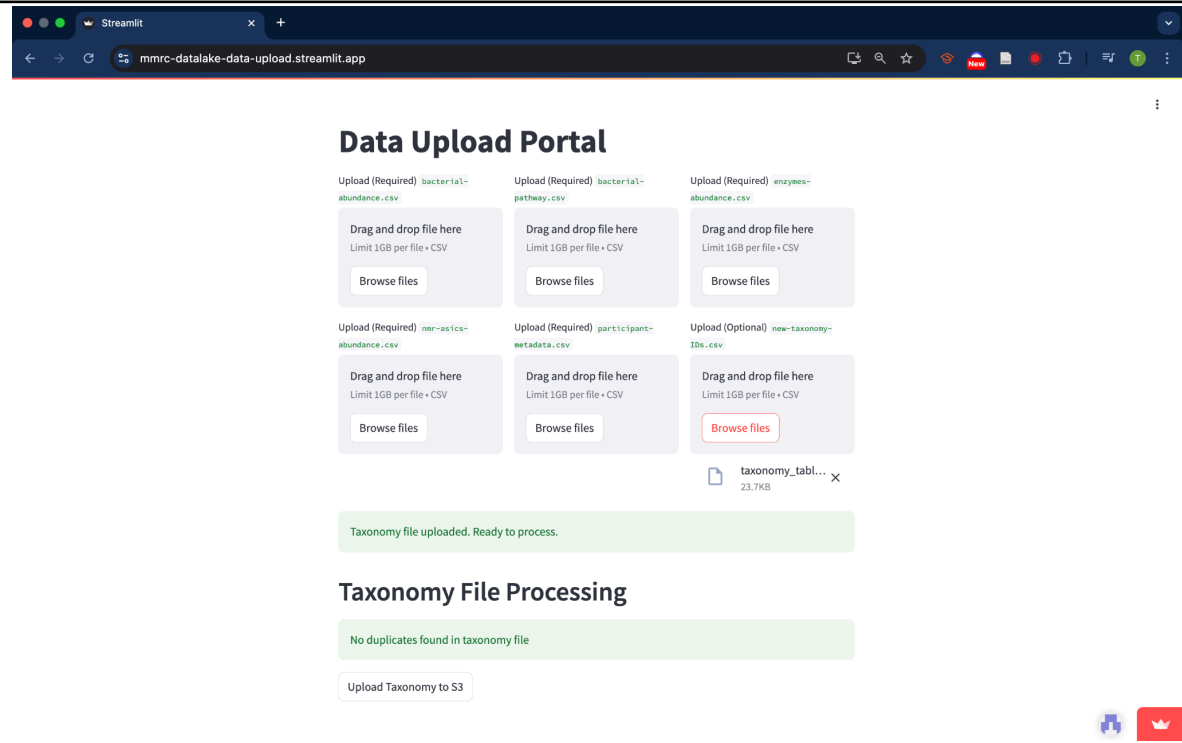
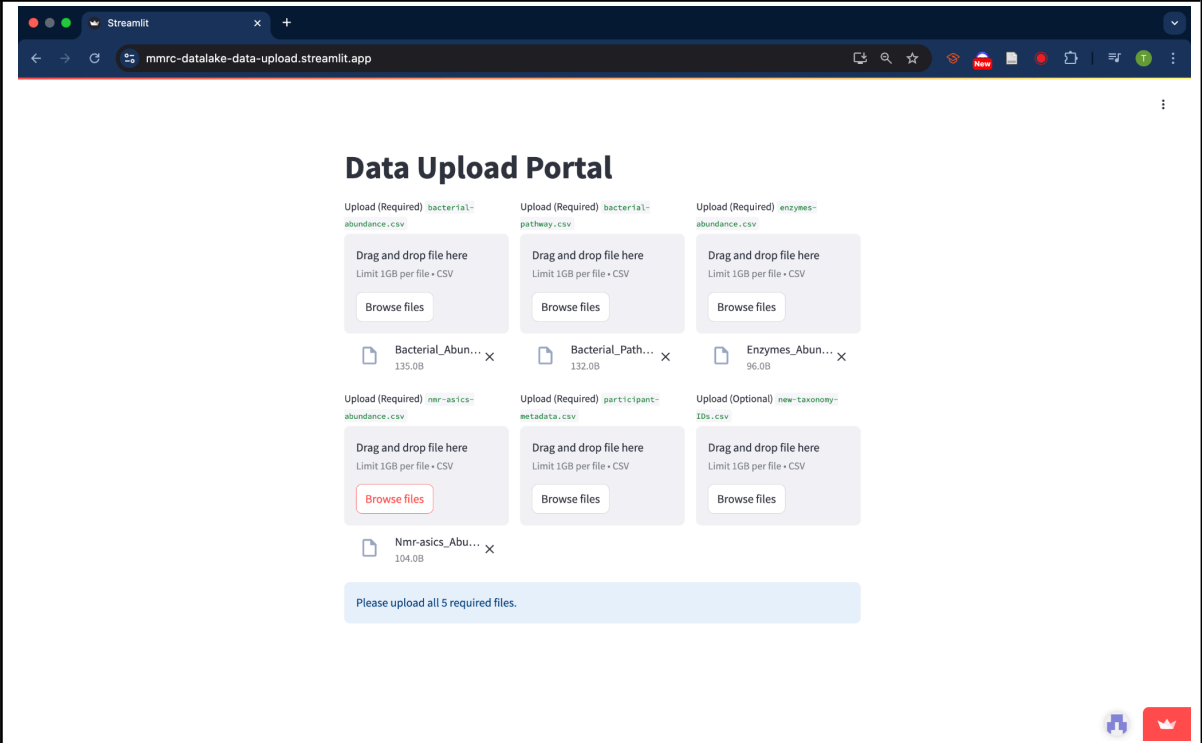
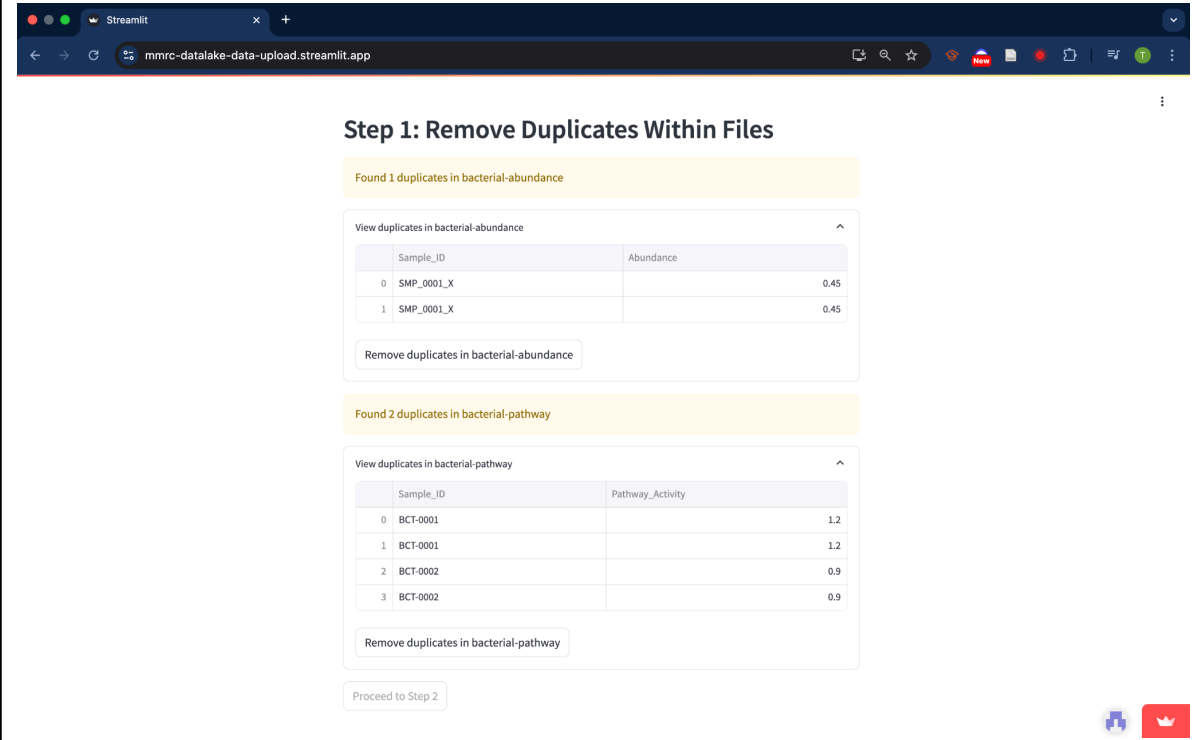
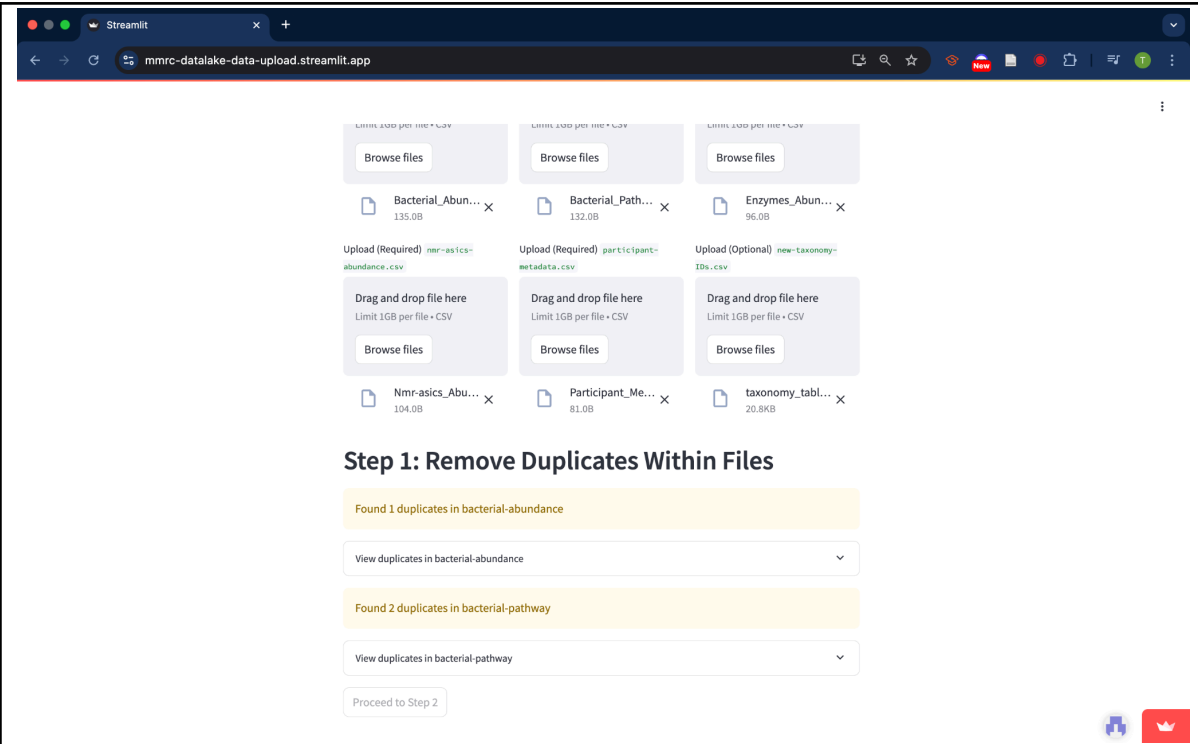
Test ID	F02
What is being tested?	Optional file upload validation
How is it tested?	Upload only new-taxonomy-IDs.csv
Input	new-taxonomy-IDs.csv
Expected Output	"Taxonomy file uploaded. Ready to process." success message and session moves to Taxonomy File Processing stage
Actual Output	Same as expected output
Output Evidence	
 The screenshot shows a web application titled 'Data Upload Portal'. It features six upload slots for different CSV files: 'bacterial-abundance.csv', 'bacterial-pathway.csv', 'enzymes-abundance.csv', 'nmr-asics-abundance.csv', 'participant-metadata.csv', and 'new-taxonomy-IDs.csv'. Each slot has a 'Drag and drop file here' instruction, a 'Limit 1GB per file • CSV' note, and a 'Browse files' button. The 'new-taxonomy-IDs.csv' slot is highlighted with a red border and a 'Browse files' button. Below the upload slots, a green message box states 'Taxonomy file uploaded. Ready to process.' and a button labeled 'Upload Taxonomy to S3' is visible. The browser's address bar shows 'mmrc-datalake-data-upload.streamlit.app'.	
Result	PASS

Figure 10: Front-end UI shows taxonomy table is ready to be uploaded

Test ID	F03
What is being tested?	Missing file validation
How is it tested?	Upload any 1-4 out of 5 required files
Input	bacterial-abundance.csv, bacterial-pathway.csv, enzymes-abundance.csv, nmr-asics-abundance.csv excluding participant-metadata.csv
Expected Output	"Please upload all 5 required files." info message

Actual Output	"Please upload all 5 required files." info message
Output Evidence	
	
<i>Figure 11: Front-end UI shows prompt to user to upload all 5 required files for validation</i>	
Result	PASS

Test ID	F04
What is being tested?	Duplicate detection within single file (Validation Step 1)
How is it tested?	Upload all of the 5 required files , 2 of which have duplicate IDs in the first column. File 1 has 1 duplicate ID and file 2 has 2 duplicate IDs
Input	Bacterial-abundance.csv (containing duplicates for 1 sample ID), bacterial-pathway.csv (containing duplicates for 2 sample IDs), enzymes-abundance.csv, nmr-asics-abundance.csv, participant-metadata.csv
Expected Output	Warning message showing count of duplicates with expandable view (<i>1 duplicate for bacterial abundance and 2 duplicates for bacterial pathway</i>)
Actual Output	Same as expected output
Output Evidence	



Figures 12 & 13: Front-end UI shows duplicates within each file identified

Result	PASS
--------	------

Test ID	F05
---------	-----

What is being tested?	Single-file duplicate removal via UI button
How is it tested?	1. Upload all of any of the required files with known duplicates 2. Click the removal button for the specific file(s) with duplicates
Input	Bacterial-abundance.csv (containing duplicates for 1 sample ID), bacterial-pathway.csv (containing duplicates for 2 sample IDs), , enzymes-abundance.csv, nmr-asics-abundance.csv, participant-metadata.csv
Expected Output	1. First file's duplicate removal: a. bacterial-abundance warning disappears b. expander collapses for bacterial abundance file (no duplicate flag) 2. Second file's duplicate removal: a. bacterial-pathway warning disappears b. expander collapses for bacterial pathway file (no duplicate flag) 3. "No duplicates found within any files" success message
Actual Output	Same as expected output

Output Evidence

Step 1: Remove Duplicates Within Files

Found 1 duplicates in bacterial-abundance

View duplicates in bacterial-abundance

	Sample_ID	Abundance
0	SMP_0001_X	0.45
1	SMP_0001_X	0.45

Remove duplicates in bacterial-abundance

Found 2 duplicates in bacterial-pathway

View duplicates in bacterial-pathway

	Sample_ID	Pathway_Activity
0	BCT-0001	1.2
1	BCT-0001	1.2
2	BCT-0002	0.9
3	BCT-0002	0.9

Remove duplicates in bacterial-pathway

Proceed to Step 2

Streamlit

mmrc-datalake-data-upload.streamlit.app

Limit 1GB per file • CSV

Browse files

Bacterial_Abu...

135.0B

×

Limit 1GB per file • CSV

Browse files

Bacterial_Path...

132.0B

×

Limit 1GB per file • CSV

Browse files

Enzymes_Abu...

96.0B

×

Upload (Required)

nmr-asics-

abundance.csv

Drag and drop file here

Limit 1GB per file • CSV

Browse files

Nmr-asics_Abu...

104.0B

×

Upload (Required)

participant-

metadata.csv

Drag and drop file here

Limit 1GB per file • CSV

Browse files

Participant_Me...

81.0B

×

Upload (Optional)

new-taxonomy-

IDs.csv

Drag and drop file here

Limit 1GB per file • CSV

Browse files

taxonomy_tabl...

20.8KB

×

Step 1: Remove Duplicates Within Files

Found 2 duplicates in bacterial-pathway

View duplicates in bacterial-pathway

Proceed to Step 2

Streamlit

mmrc-datalake-data-upload.streamlit.app

135.0B

Limit 1GB per file • CSV

Browse files

Nmr-asics_Abu...

104.0B

×

132.0B

Limit 1GB per file • CSV

Browse files

Participant_Me...

81.0B

×

96.0B

Limit 1GB per file • CSV

Browse files

taxonomy_tabl...

20.8KB

×

Step 1: Remove Duplicates Within Files

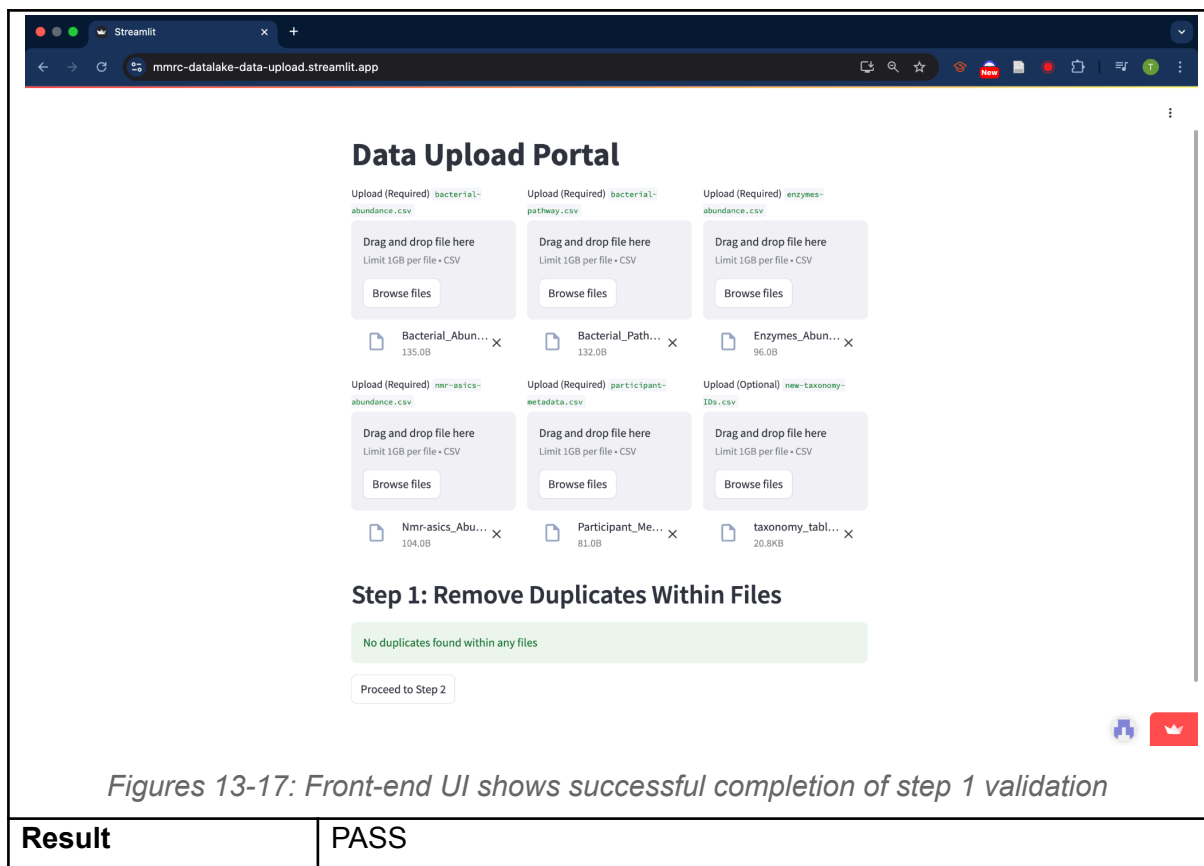
Found 2 duplicates in bacterial-pathway

View duplicates in bacterial-pathway

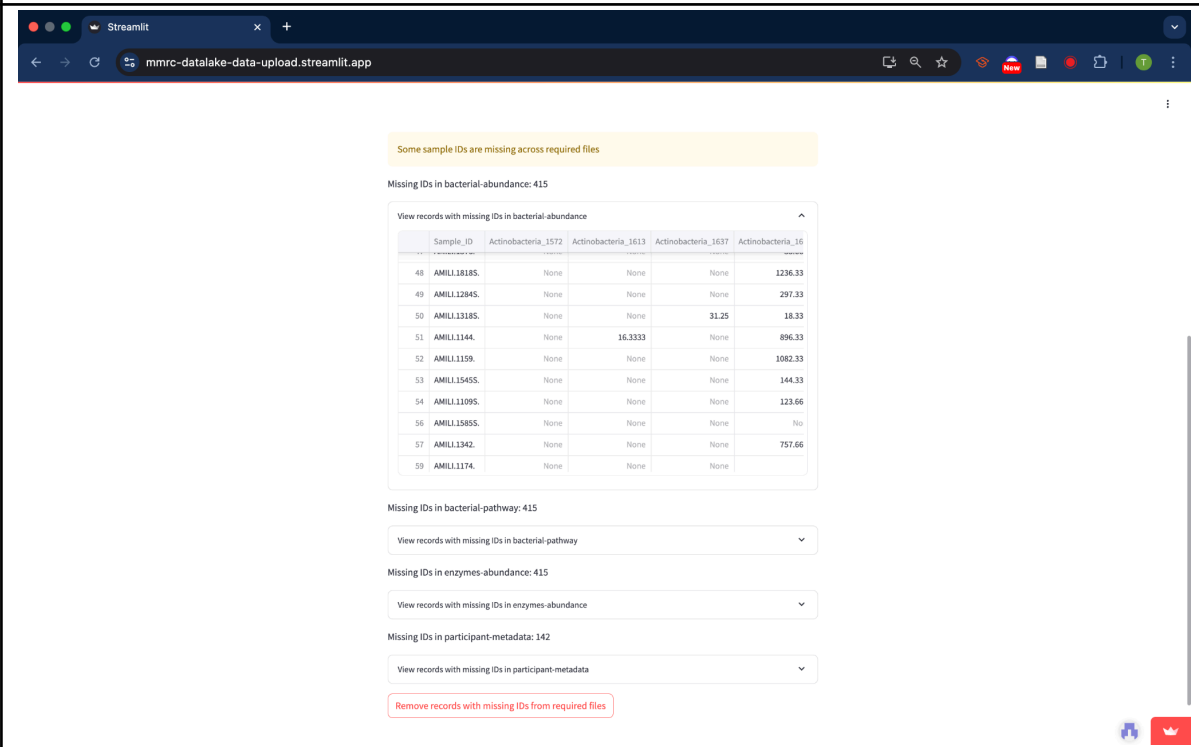
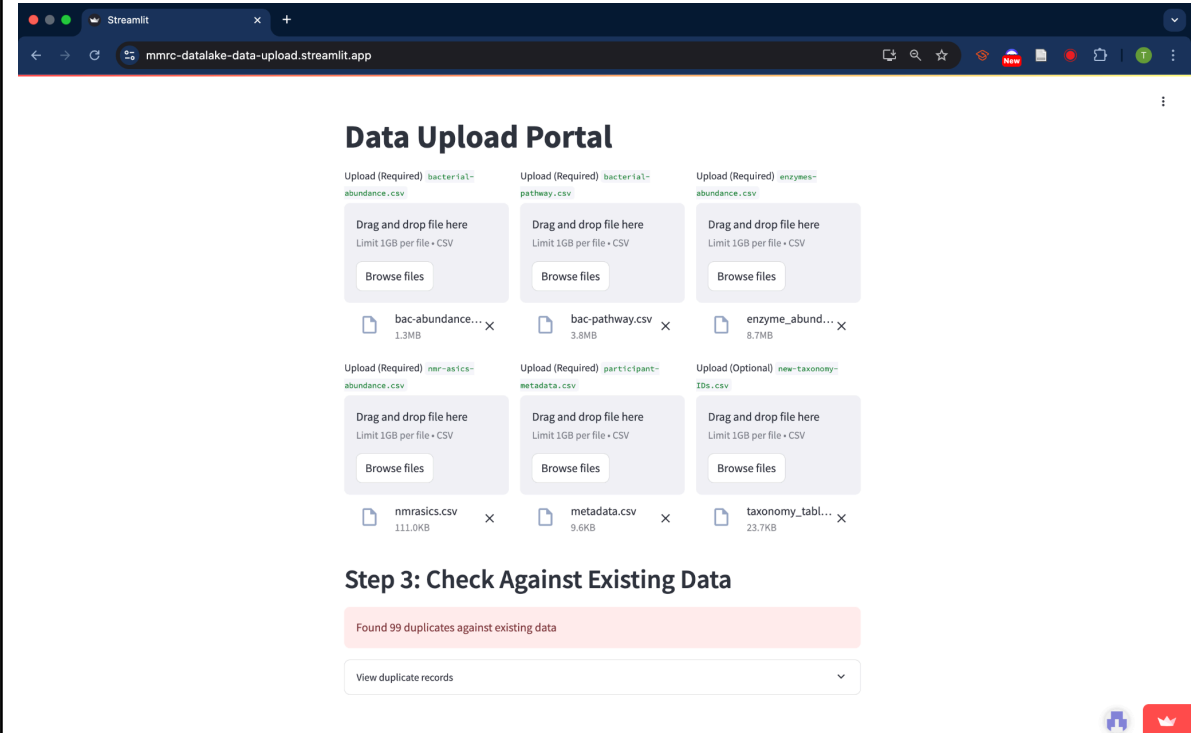
	Sample_ID	Pathway_Activity	
0	BCT-0001		1.2
1	BCT-0001		1.2
2	BCT-0002		0.9
3	BCT-0002		0.9

Remove duplicates in bacterial-pathway

Proceed to Step 2



Test ID	F06
What is being tested?	Cross-file record completeness enforcement (Validation Step 1)
How is it tested?	1. Upload all 5 required files with some non-overlapping sample IDs 2. Verify bulk removal of records of non-overlapping sample IDs when clicking "Remove records with missing IDs" 3. Verify automatic transition to next validation step
Input	All 5 required files where some sample IDs exist in some files but not in others
Expected Output	1. Warning about missing IDs with expandable view of affected records and a button to bulk drop the affected records 2. Upon clicking of the button for bulk removal, warning message disappears 3. Progress advances to Step 3 ("Check Against Existing Data")

Actual Output	Same as expected output
Output Evidence 	
 Step 3: Check Against Existing Data Found 99 duplicates against existing data View duplicate records	
Result	PASS

Test ID	F07
What is being tested?	Duplicate check against DynamoDB (Validation Step 3)
How is it tested?	Upload all required files containing sample IDs that already exist in DynamoDB lookup table
Input	bacterial-abundance.csv, bacterial-pathway.csv, enzymes-abundance.csv, nmr-asics-abundance.csv, participant-metadata.csv (whose complete records have already been uploaded to S3 before)
Expected Output	Error message showing count of duplicates with expandable view of metadata file (master reference file for incoming complete records) and a button to bulk remove the duplicates <i>(Note: Clicking of the bulk removal button will lead to the scenario in Test ID F07)</i>
Actual Output	Error message showing count of duplicates with expandable view of metadata file (master reference file for incoming complete records) and a button to bulk remove the duplicates

Output Evidence

Step 3: Check Against Existing Data

Found 99 duplicates against existing data

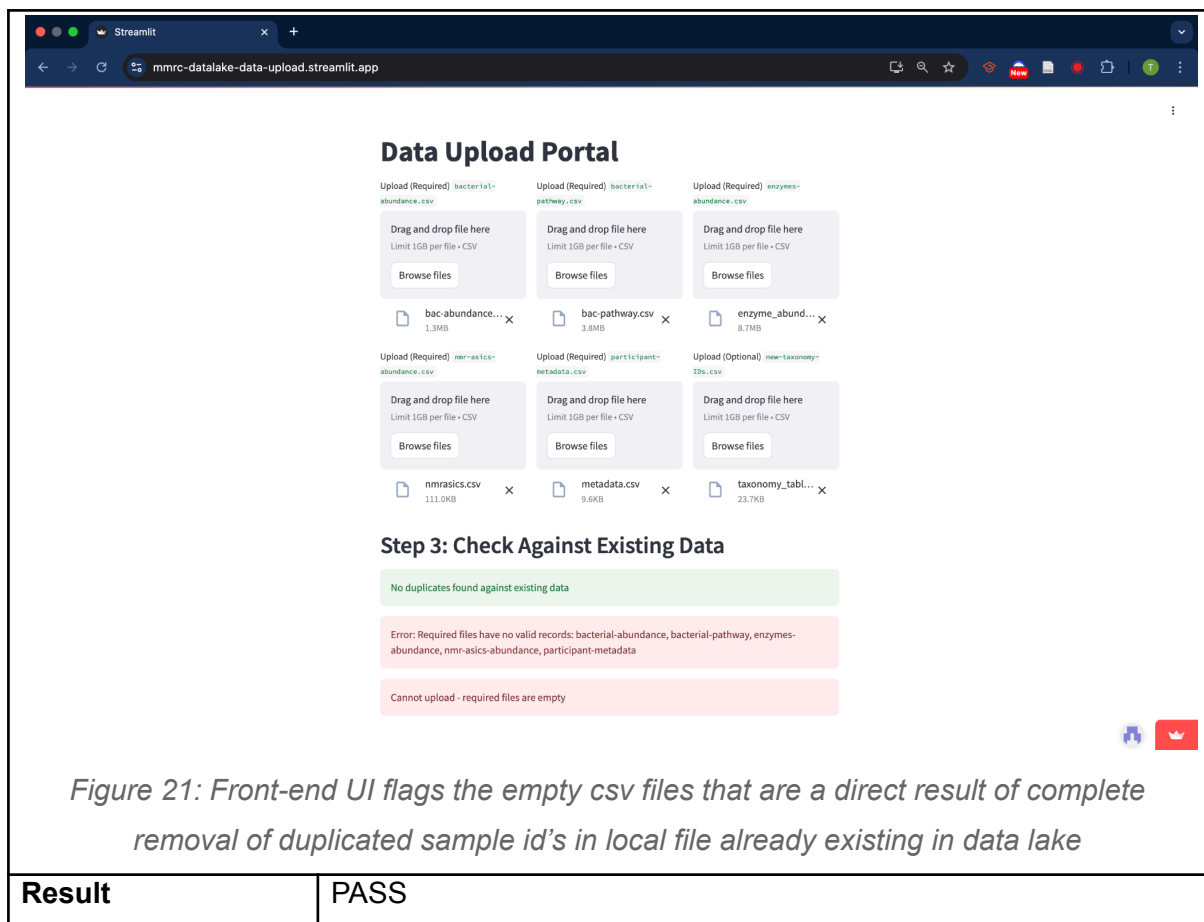
Study ID	Gender	BY	Age	AgeGroup	Ethnicity	State	Sago	GPAQ	PSQI	
146	5005	F	1953	70	E	O	Mukah	A	active	4
147	5006	F	1965	58	D	O	Mukah	A	Inadequate	3
148	5007	F	1956	67	E	O	Mukah	A	active	10
149	5008	M	1953	70	E	O	Mukah	B	active	8
150	5009	M	1957	66	E	O	Sarawak	A	Inadequate	2
151	5010	F	1959	64	E	O	Sarawak	A	Inadequate	3
152	5011	M	1984	39	B	O	Mukah	B	Inadequate	4
153	5012	M	1978	45	C	O	Sarawak	A	Inadequate	4
154	5013	M	1977	46	C	O	Mukah	A	active	11
155	5014	M	1995	28	A	O	Sarawak	A	active	5

Remove all duplicate entries from required files

Figure 20: Front-end UI shows identification of duplicated sample id's in local file already existing in data lake

Result	PASS
---------------	------

Test ID	F07
What is being tested?	Pre-upload empty file validation
How is it tested?	Process the required files through the first two validation steps. At step 3, check for duplicate entries by comparing the records against the DynamoDB lookup table. When duplicates are identified, remove them in bulk. Since all the complete records have already been uploaded to the datalake previously, this removal will result in all records being deleted. Consequently, all the filtered files will be left empty.
Input	bacterial-abundance.csv, bacterial-pathway.csv, enzymes-abundance.csv, nmr-asics-abundance.csv, participant-metadata.csv (<i>whose complete records have already been uploaded to S3 before</i>)
Expected Output	Error message about required files being empty after processing, upload to s3 button isn't shown
Actual Output	Error message about required files being empty after processing, upload to s3 button isn't shown
Output Evidence	



Test ID	F09
What is being tested?	Record count reporting before upload
How is it tested?	Complete all validations with valid files
Input	5 required files (+ optional taxonomy file) with: 99 unique common records in required files 120 additional records in taxonomy file
Expected Output	Clear display showing: "Total complete records to upload: 219" "Complete records in required files: 99" "Records in optional file(s): 120"
Actual Output	Same as expected output
Output Evidence	

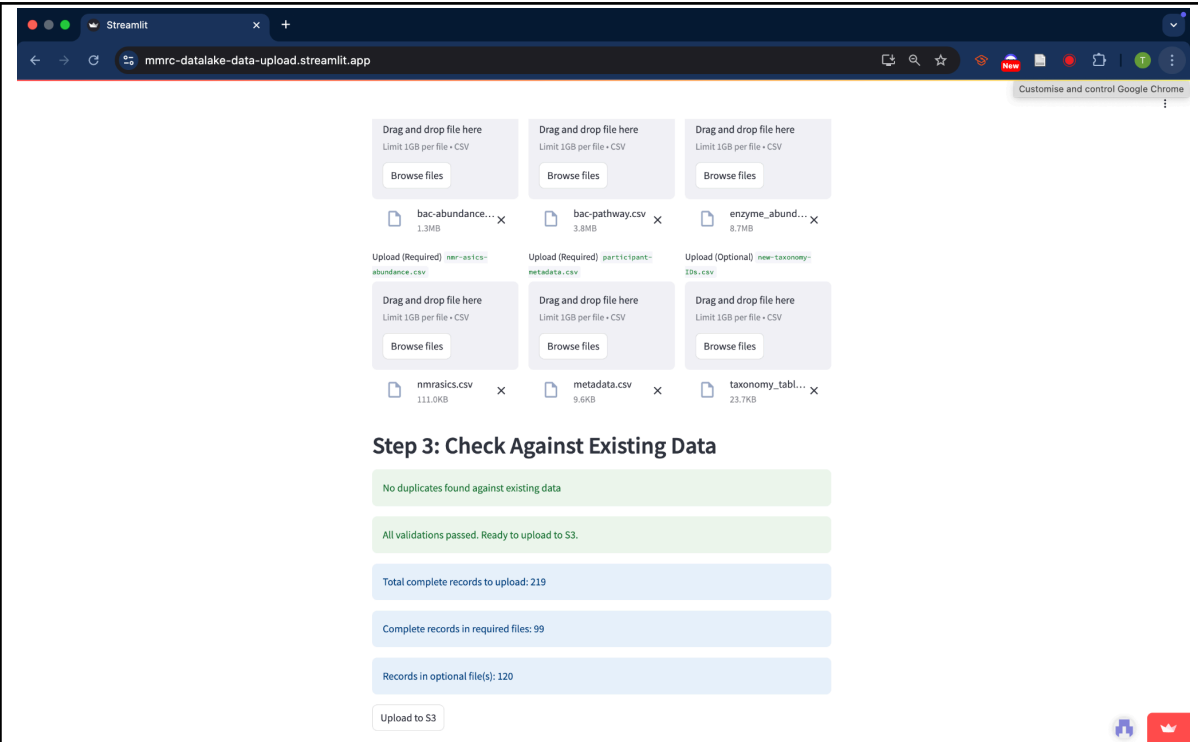


Figure 22: Front-end UI shows completion of all 3 validation steps where files are now ready to be uploaded to the data lake

Result	PASS
--------	------

Test ID	F10
What is being tested?	Successful upload confirmation
How is it tested?	Complete all validations with valid files and click upload to S3 button
Input	Valid set of required files passing all checks
Expected Output	Successful upload message with S3 confirmation
Actual Output	Same as expected output
Output Evidence	

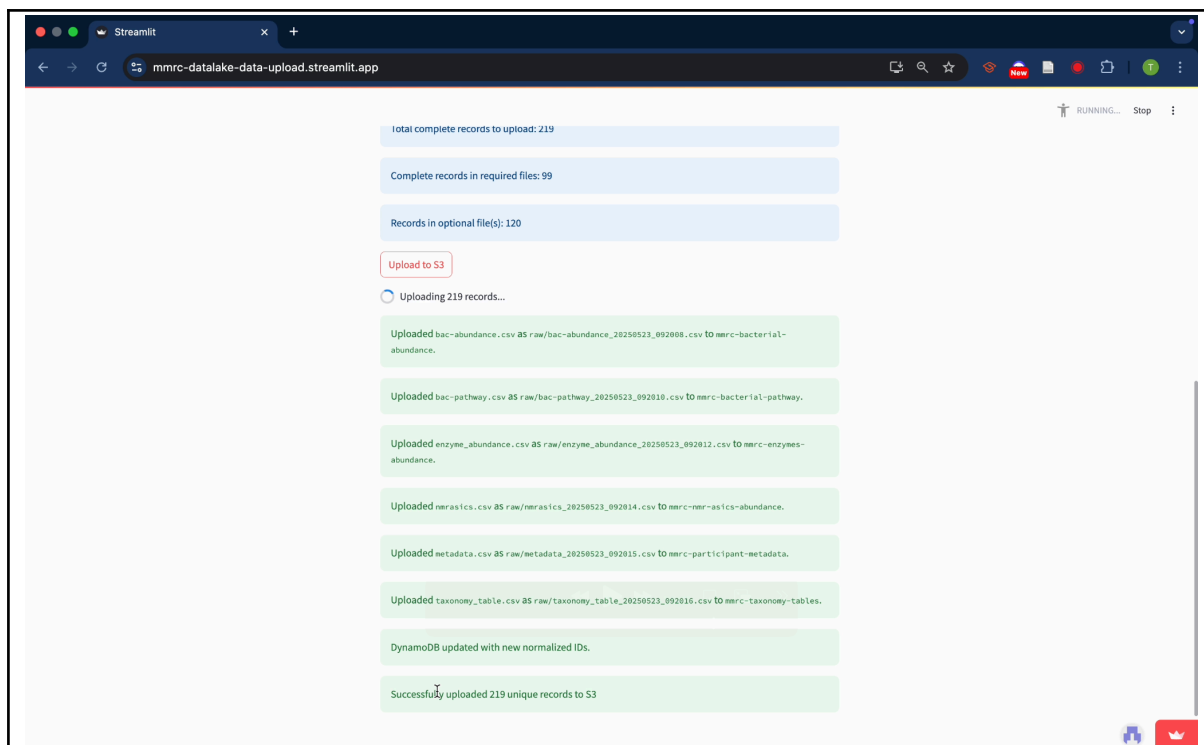
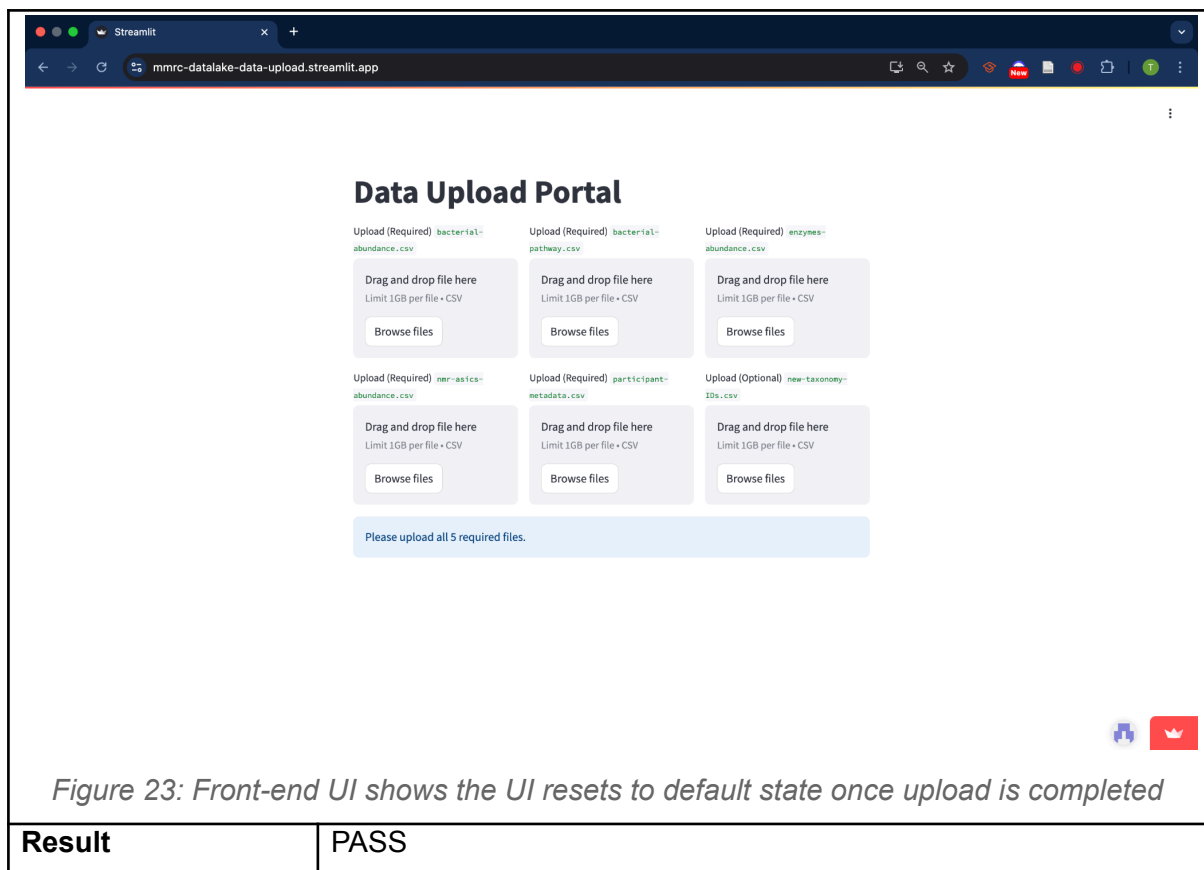


Figure 22: Front-end UI shows completed upload of all csv files to the data lake and insertion of new hashed sample id's to DynamoDB table

Result	PASS
--------	------

Test ID	F11
What is being tested?	Session state reset after upload
How is it tested?	Complete upload and observe UI reset
Input	Successful upload completion
Expected Output	UI resets to initial state with empty file uploaders (continuation from scenario in Test ID F10)
Actual Output	Same as expected output
Output Evidence	



Test ID	F12
What is being tested?	Mid-process file removal detection and session restart
How is it tested?	1. Upload all 5 required files 2. Complete Step 1 and Proceed to Step 2 3. Remove one of the uploaded file during processing
Input	1. Initial set: All 5 required files 2. During Step 2: Remove bacterial-abundance.csv
Expected Output	System detects file change and resets to Step 1
Actual Output	Same as expected output
Output Evidence	

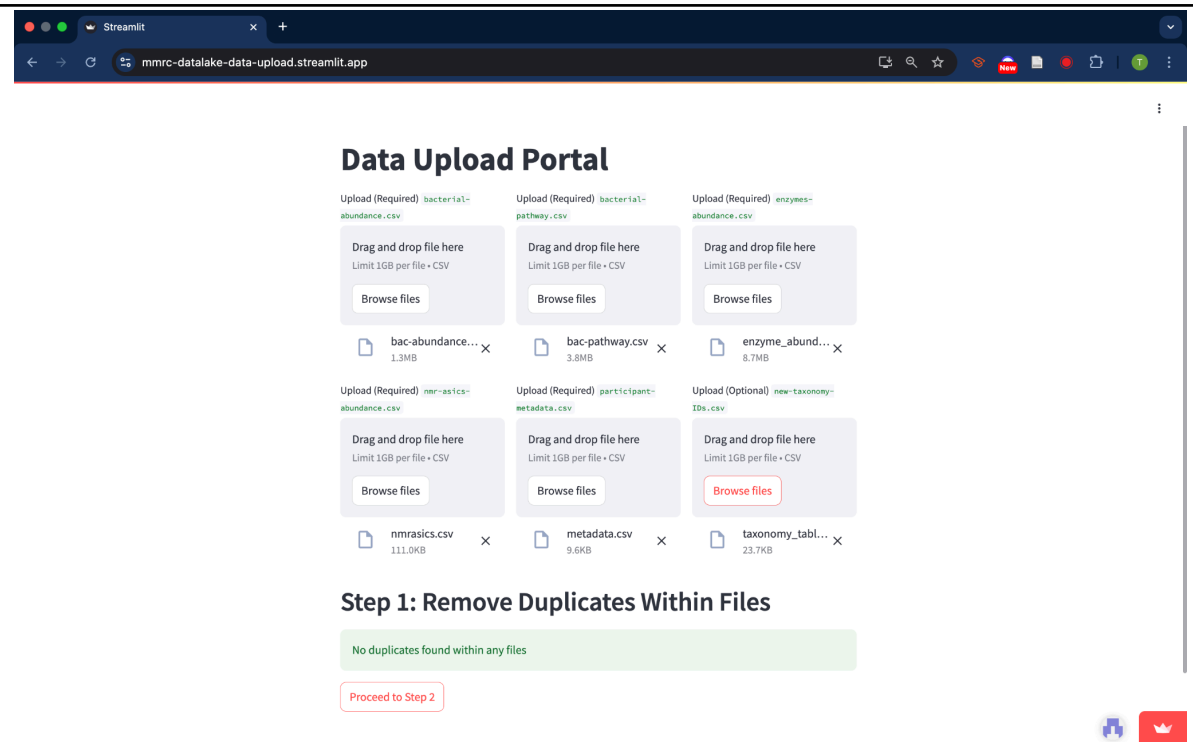


Figure 24: All the uploaded files are ready for processing and the session is now proceeding to Step 2 of the validation upon button click

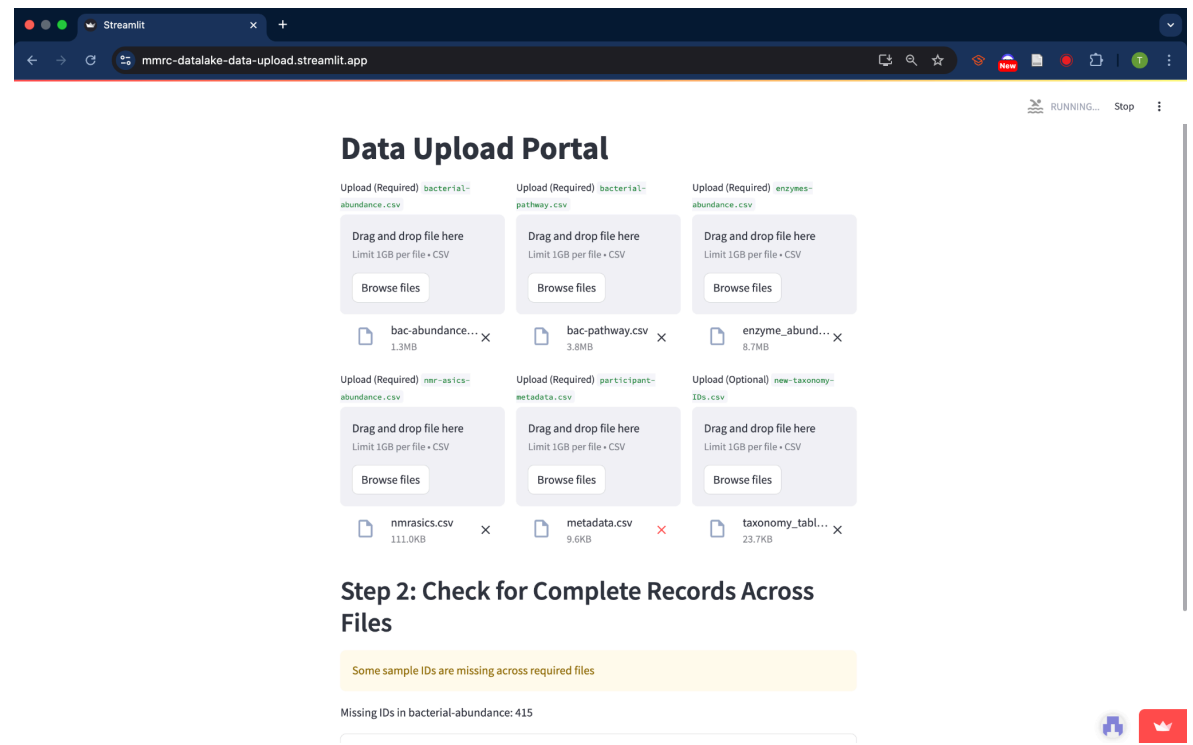


Figure 25: The metadata file is being removed mid process at Step 2 of the validation

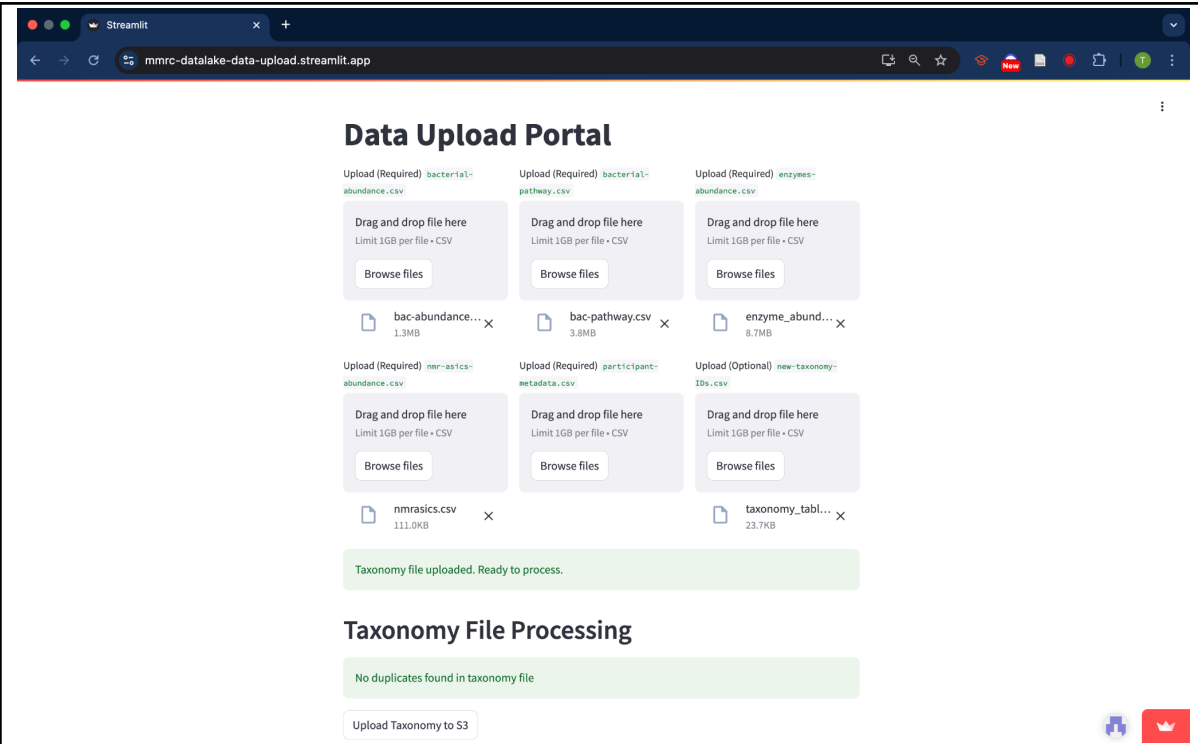


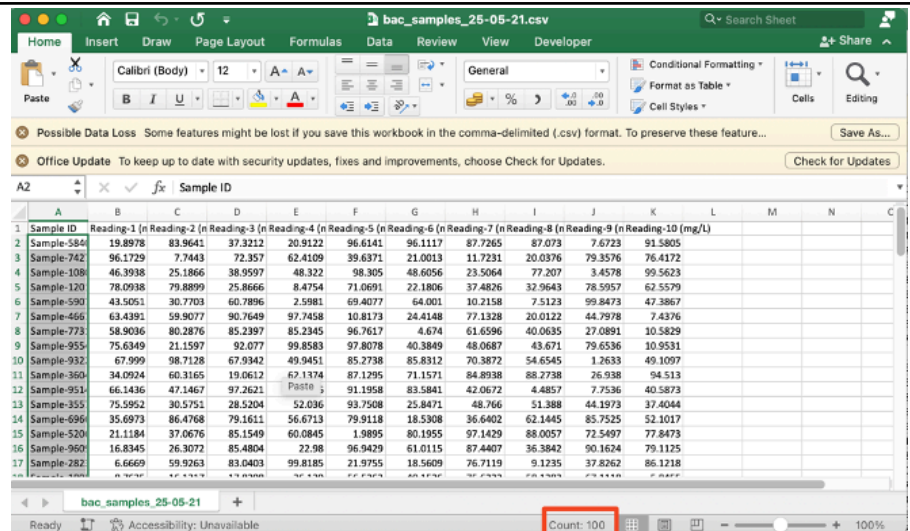
Figure 26: When the metadata file was removed, the session got reset and restarted. The session then detected that the requirement of all 5 required files needing to be uploaded has now become false hence the system only approves the processing of the optional taxonomy file that's readily uploaded

Result	PASS
--------	------

4.2 AWS Cloud Testing (BackEnd)

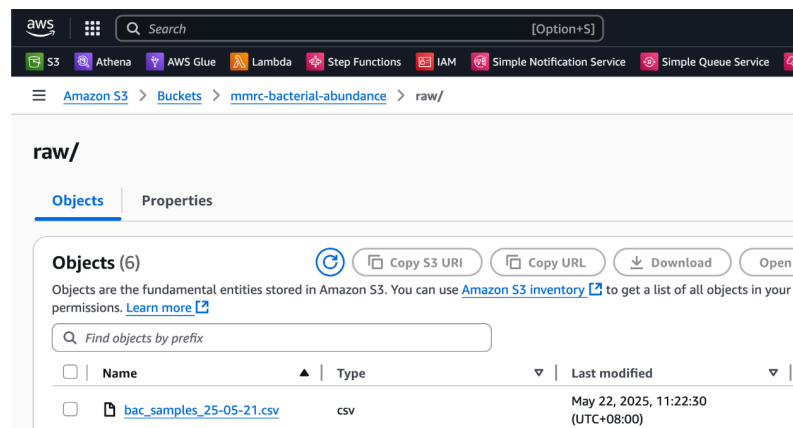
Test ID	B01
What is being tested?	Effectiveness of data lake to handle data ingestion and transformation for 1 csv file uploaded to a single source S3 bucket.
How is it tested?	1. Upload csv file to the raw subfolder within its associated source bucket. 2. Wait for an AWS Glue job start execution email notification to indicate the start of the data transformation process. 3. Wait for job completion email notification. 4. Query the specific Iceberg table on AWS Athena to confirm data has been added.
Input	1 correctly formatted csv file.
Expected Output	Results of querying the associated Iceberg table shows all the data from the uploaded csv file has been successfully inserted.
Actual Output	All data from uploaded csv is returned when querying the table
Result	PASS

Screenshot Evidence



Sample ID	Reading-1 (n)	Reading-2 (n)	Reading-3 (n)	Reading-4 (n)	Reading-5 (n)	Reading-6 (n)	Reading-7 (n)	Reading-8 (n)	Reading-9 (n)	Reading-10 (mg/L)
1 Sample ID	19.8978	83.9641	37.3212	20.8122	96.6141	96.1117	87.7265	87.073	7.6723	91.5805
2 Sample-584	96.1729	7.7443	72.357	62.4109	39.6371	21.0013	11.7231	20.0376	79.3576	76.4172
3 Sample-742	46.3938	25.1866	38.9597	48.322	98.305	48.6056	23.5064	77.207	3.4578	99.5623
4 Sample-120	78.0938	79.8899	25.8666	8.4754	71.0691	22.1806	37.4826	32.9643	78.5957	62.5579
5 Sample-590	43.5051	30.7703	60.7896	2.5981	69.4077	64.001	10.2158	7.5123	99.8473	47.3867
6 Sample-466	63.4391	59.9077	90.7649	97.7458	10.8173	24.4148	77.1328	20.0122	44.7978	7.4376
7 Sample-773	58.9036	80.2876	85.2397	85.2345	96.7617	4.674	61.6596	40.0635	27.0891	10.5829
8 Sample-955	75.6349	21.1597	92.077	99.8583	97.8078	40.3849	48.0687	43.671	79.6536	10.9531
9 Sample-932	67.999	98.7128	67.9342	49.9451	85.2738	85.8312	70.3872	54.6545	1.2633	49.1097
10 Sample-360	34.0924	60.3165	19.0612	62.1174	87.1295	71.1571	84.8938	88.2738	26.938	94.513
11 Sample-951	66.1436	47.1467	97.2621	91.1958	83.5841	42.0672	4.4857	7.7536	40.5873	40.5873
12 Sample-355	75.5952	30.5751	28.5204	52.036	93.7508	25.8471	48.766	51.388	44.1973	37.4044
13 Sample-696	35.6973	86.4768	79.1611	56.6713	79.9118	18.5308	36.6402	62.1445	85.7525	52.1017
14 Sample-520	21.1184	37.0676	85.1549	60.0845	1.9895	80.1955	97.1429	88.0057	72.5497	77.8473
15 Sample-960	16.8345	26.3072	85.4804	22.98	96.9429	61.0115	87.4407	36.3842	90.1624	79.1125
16 Sample-282	6.6669	59.9263	83.0403	99.8185	21.9755	18.5609	76.7119	9.1235	37.8262	86.1218

Figure 27: The first 16 rows of a csv file with 100 rows to be uploaded



Name	Type	Last modified
bac_samples_25-05-21.csv	csv	May 22, 2025, 11:22:30 (UTC+08:00)

Figure 28: Timestamp of csv file uploaded to AWS S3 raw folder

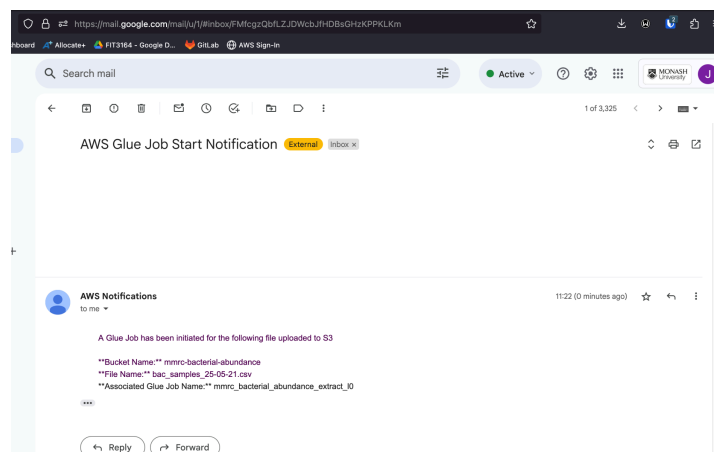


Figure 29: AWS Glue job start execution email notification for the uploaded csv file

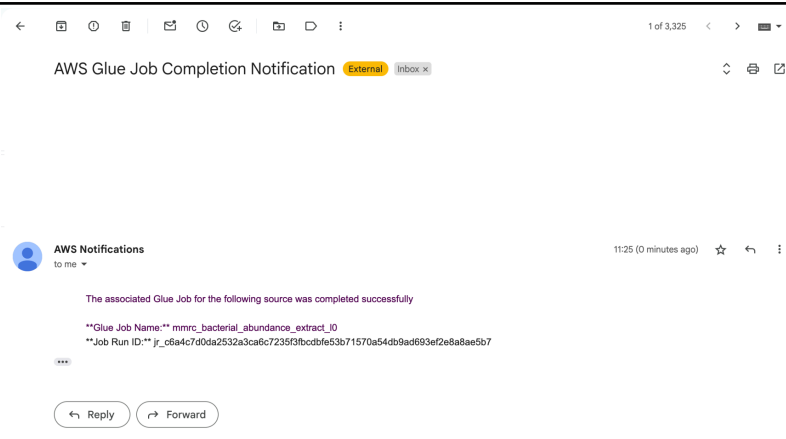


Figure 30 shows the AWS Glue job completion email notification for the uploaded csv file

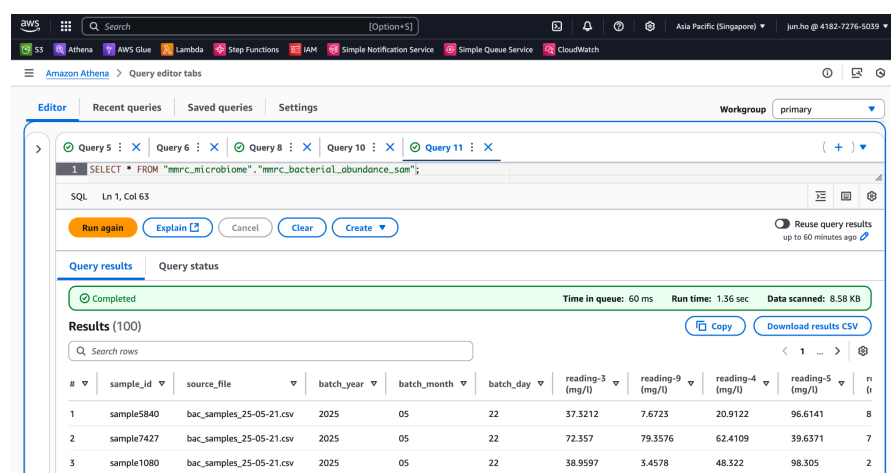
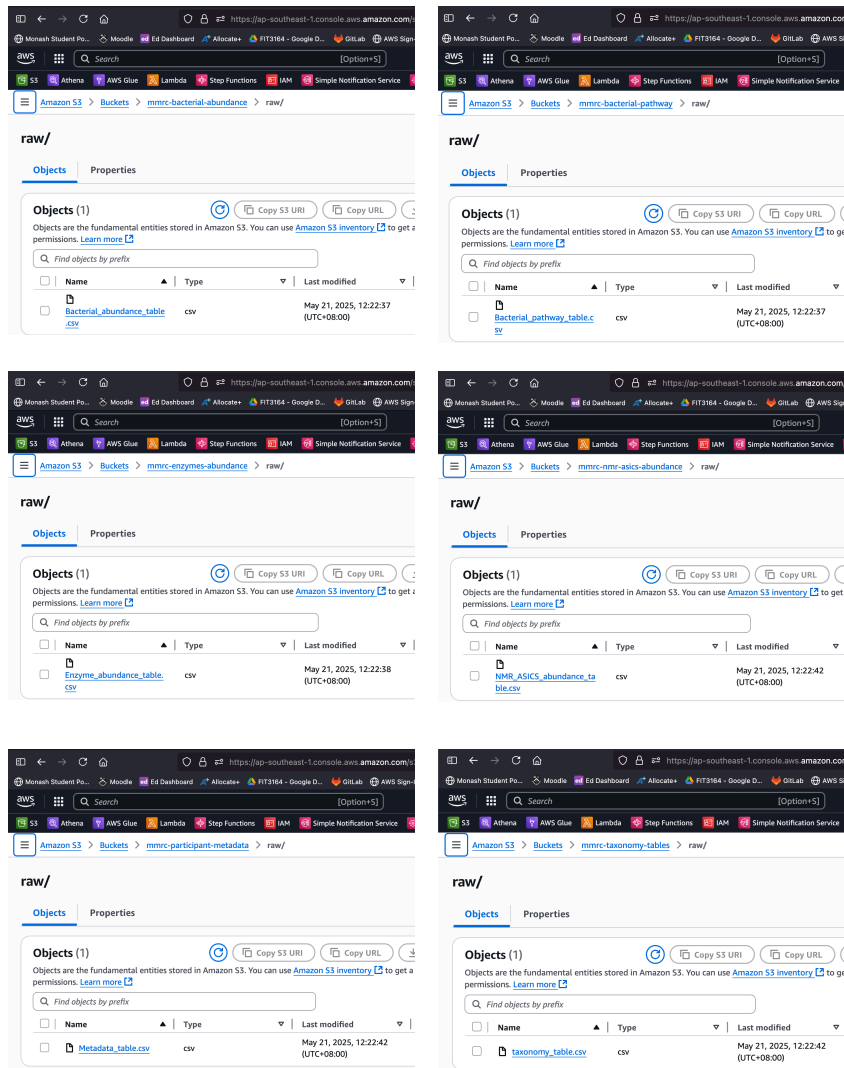


Figure 31: 100 rows of data from uploaded csv file successfully inserted into its associated Iceberg table

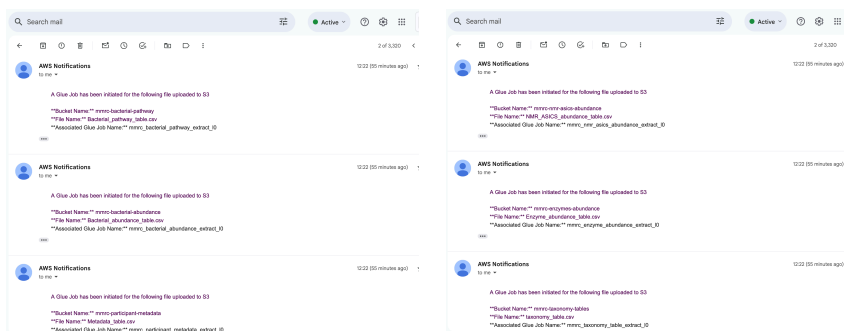
Test ID	B02
What is being tested?	Effectiveness of data lake to handle ingestion and transformation of 1 csv file uploaded to the raw subfolder of each available source bucket.

How is it tested?	1. Upload 1 csv file to the raw subfolder of each available source bucket. a. NOTE: Previous test was done with a sample dataset created by the team. This test uses actual sample data provided by the Microbiome Research team. Though the data types and file structure between these 2 groups of sample files are the same. 2. Wait for an AWS Glue job start execution email notification to indicate the start of the data transformation process. 3. Wait for job completion email notification for each job started. 4. Query all Iceberg tables on AWS Athena to confirm data has been added.
Input	A csv file to upload for each available S3 source bucket.
Expected Output	All Iceberg tables on Athena will have all data from their respective csv files successfully inserted into the table.
Actual Output	All data from all uploaded csv files are returned when querying the respective Iceberg tables
Result	PASS

Screenshot Evidence



Figures 32-37: Upload timestamps of 1 csv file uploaded per source bucket to simulate bulk upload



Figures 38 & 39: 6 AWS Glue job start emails received for each file uploaded

Amazon Athena Query Editor interface showing Query 11 results.

Query 11: `SELECT * FROM "mmrc_microbiome"."mmrc_enzyme_abundance_sam" ;`

Query results: Completed. Time in queue: 58 ms. Run time: 3.155 sec. Data scanned: 9.07 MB. Results (1,022).

#	sample_id	source_file	batch_year	batch_month	batch_day	6.3.3.1	2.3.1.233	5.4.99.18	4.1.1.44
1	sample50235	Enzyme_abundance_table.csv	2025	05	21	18568.71793	0.0	27224.90842	450.625
2	sample13715	Enzyme_abundance_table.csv	2025	05	21	36060.16519	0.0	41187.61042	5.35

Amazon Athena Query Editor interface showing Query 11 results.

Query 11: `SELECT * FROM "mmrc_microbiome"."mmrc_nmr_omics_abundance_sam" ;`

Query results: Completed. Time in queue: 67 ms. Run time: 2.208 sec. Data scanned: 12.55 KB. Results (256).

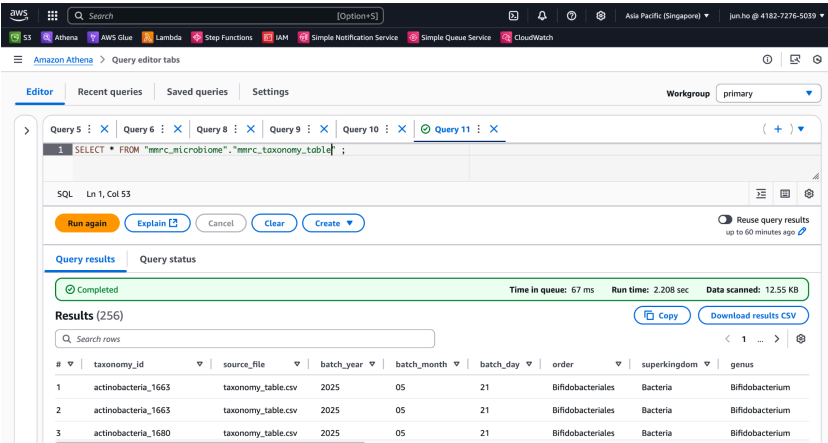
#	taxonomy_id	source_file	batch_year	batch_month	batch_day	order	superkingdom	genus
1	actinobacteria_1663	taxonomy_table.csv	2025	05	21	Bifidobacteriales	Bacteria	Bifidobacterium
2	actinobacteria_1663	taxonomy_table.csv	2025	05	21	Bifidobacteriales	Bacteria	Bifidobacterium
3	actinobacteria_1680	taxonomy_table.csv	2025	05	21	Bifidobacteriales	Bacteria	Bifidobacterium

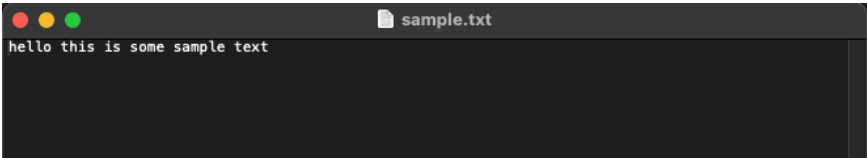
Amazon Athena Query Editor interface showing Query 11 results.

Query 11: `SELECT * FROM "mmrc_microbiome"."mmrc_participant_metadata" ;`

Query results: Completed. Time in queue: 56 ms. Run time: 1.288 sec. Data scanned: 5.51 KB. Results (482).

#	sample_id	source_file	batch_year	batch_month	batch_day	age	agegroup	ethnicity	sago	by	gpaq
1	sample1115	Metadata_table.csv	2025	05	21	29	A	C	NA	1993	active
2	sample1115	Metadata_table.csv	2025	05	21	29	A	C	NA	1993	active
3	sample1117	Metadata_table.csv	2025	05	21	29	A	M	NA	1993	Inadequat

	 <i>Figures 42-47: Query results that for each uploaded csv file, all data has been successfully inserted into its associated Iceberg Table</i>
--	--

Test ID	B03
What is being tested?	Validate event driven processing pipeline is not triggered when a file in any format other than csv is uploaded.
How is it tested?	1. Upload a txt file, excel file and a json file with mock data directly into a S3 raw folder to mimic an upload error by a user that bypasses upload via the Streamlit front end.
Input	1 txt file, 1 excel file, 1 json file all containing mock data
Expected Output	Files are all uploaded successfully but no Glue Job Start Notification email is sent
Actual Output	All files uploaded successfully and no job start email received
Result	PASS
Screenshot Evidence	

	A	B	C	D	E	F	G	H	I
1	Sample_ID	Test_Col_1	Test_Col_2						
2	Sam_1001	12.43	2.23						
3	Sam_1002	23.43	4.21						
4	Sam_1003	5.2	2.63						
5	Sam_1004	23.12	7.334						
6	Sam_1005	67.45	634.77						
7	Sam_1006	765.4	845.46						
8	Sam_1007	65.33	43.23						
9	Sam_1008	8.43	465.34						
10	Sam_1009	78.6	345.32						
11	Sam_1010	99.3	352.32						
12									
13									
14									
15									

```

1  {
2    "sample": {
3      "id": "sample_001",
4      "collection_date": "2025-05-22",
5      "location": "gut",
6      "subject_id": "subject_123",
7      "study_group": "control"
8    },
9    "taxonomy_level": "genus",
10   "abundance": [
11     {
12       "taxon": "Bacteroides",
13       "relative_abundance": 35.2,
14       "rank": "genus"
15     },
16     {
17       "taxon": "Faecalibacterium",
18       "relative_abundance": 22.1,
19       "rank": "genus"
20     },
21     {
22       "taxon": "Prevotella",
23       "relative_abundance": 14.6,
24       "rank": "genus"
25     },
26     {
27       "taxon": "Escherichia",
28       "relative_abundance": 5.9,
29       "rank": "genus"
30     },
31     {
32       "taxon": "Others",
33       "relative_abundance": 22.2,
34       "rank": "other"
35     }
36   ],
37   "notes": "Sample passed quality control. Sequencing depth: 5M reads."
38 }
39

```

Figures 48-50: 3 files with mock data, a txt, json and excel file, none of which will trigger execution of the event driven data transformation pipeline.

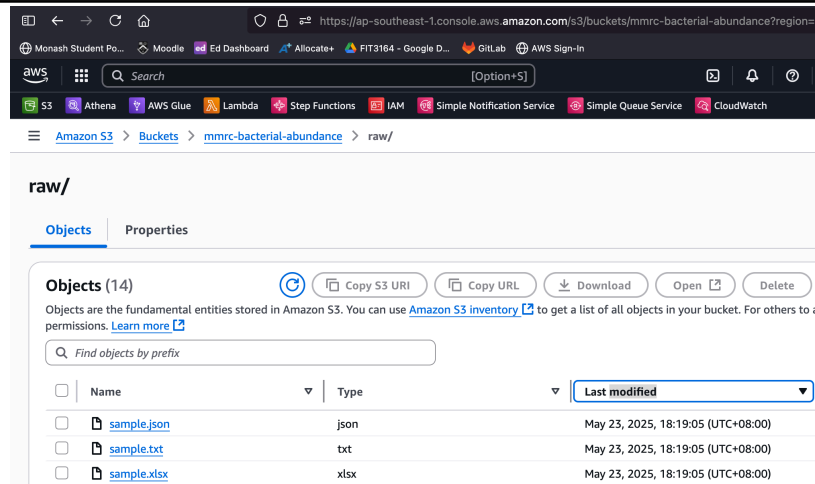


Figure 51: Upload timestamps for 3 files successfully uploaded

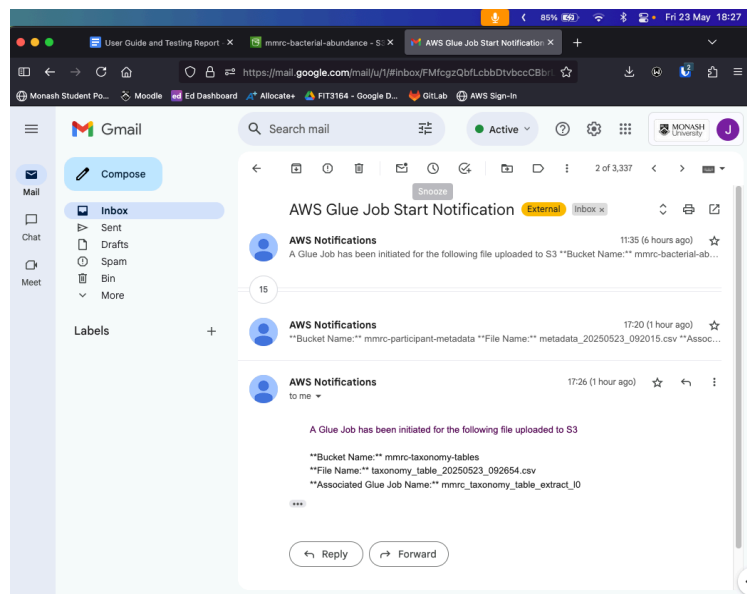


Figure 52: No job start email notifications have been received after 5 minutes, indicating the transformation pipeline was not triggered

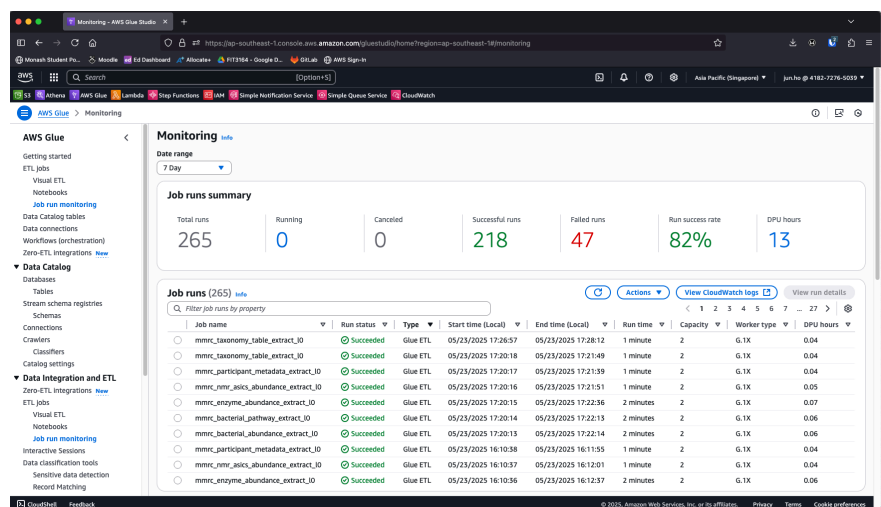


Figure 53: Glue Job Monitoring Console shows no Job executions triggered

Section 5: Integration Test

Test Case ID	IT01
What is being tested	End-to-end upload and validation flow for five CSVs.
How is it tested	In the Streamlit UI, drag-and-drop the five files; verify real-time checks pass; click “Upload to S3.”
Input	Five CSVs (e.g. bacterial-abundance.csv, etc.), each with header Sample ID and Study ID. DynamoDB is empty at start.
Expected output	• UI shows “All validations passed” and “Ready to upload to S3” after verifying all the real-time checks. • Received two types of SNS notifications titled “AWS Glue Job Start Notification” and “AWS Glue Job Completion Notification.” • All five files appear under their S3 buckets in the raw subfolder. • DynamoDB table PK_Hash_Index is populated.
Actual output	Matches expected output.
Result	Pass
Screenshot	Data Upload Portal Upload (Required) <code>bacterial-abundance.csv</code> Drag and drop file here Limit 1GB per file • CSV Browse files Upload (Required) <code>bacterial-pathway.csv</code> Drag and drop file here Limit 1GB per file • CSV Browse files Upload (Required) <code>enzymes-abundance.csv</code> Drag and drop file here Limit 1GB per file • CSV Browse files Upload (Required) <code>nmr-asics-abundance.csv</code> Drag and drop file here Limit 1GB per file • CSV Browse files Upload (Required) <code>participant-metadata.csv</code> Drag and drop file here Limit 1GB per file • CSV Browse files Upload (Optional) <code>new-taxonomy-IDs.csv</code> Drag and drop file here Limit 1GB per file • CSV Browse files Please upload all 5 required files.

Figure 54 showing step 1: The application opens to a clean landing screen with six clearly labeled upload slots.

Data Upload Portal

Upload (Required) bacterial-abundance.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

 Bacterial abund...
1.2MB

×

Upload (Required) bacterial-pathway.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

 Bacterial path...
3.6MB

×

Upload (Required) enzymes-abundance.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

 Enzyme abund...
8.3MB

×

Upload (Required) nmr-asics-abundance.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

 NMR ASICS abu...
101.8KB

×

Upload (Required) participant-metadata.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

 Metadata table...
9.6KB

×

Upload (Optional) new-taxonomy-IDs.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

All required files uploaded. Processing...

Figure 55 showing step 2: drag and drop the five required CSVs into their corresponding fields. The UI immediately shows “All required files uploaded. Processing...”

Step 1: Remove Duplicates Within Files

No duplicates found within any files

Proceed to Step 2

Figure 56 showing step 3: The system scans each CSV for duplicate sample IDs and finds none. A “Proceed to Step 2” button becomes available.

Step 2: Check for Complete Records Across Files

Some sample IDs are missing across required files

Missing IDs in bacterial-abundance: 415

View records with missing IDs in bacterial-abundance

Missing IDs in bacterial-pathway: 415

View records with missing IDs in bacterial-pathway

Missing IDs in enzymes-abundance: 415

View records with missing IDs in enzymes-abundance

Missing IDs in participant-metadata: 142

View records with missing IDs in participant-metadata

Remove records with missing IDs from required files

Figure 57 showing step 4: the app checks all files against each other and reports: “Some sample IDs are missing across required files”, then click “Remove records with missing IDs from required files” to proceed to the next step.

Step 3: Check Against Existing Data

No duplicates found against existing data

All validations passed. Ready to upload to S3.

Total complete records to upload: 99

Complete records in required files: 99

Records in optional file(s): 0

Upload to S3

Figure 58 showing step 5: A concise summary of each cleaned file is displayed, and click “Upload to S3” to start the ingestion.

Uploaded Bacterial abundance table.csv as raw/Bacterial_abundance_table_20250523_081029.csv to mmrc-bacterial-abundance.

Uploaded Bacterial pathway table.csv as raw/Bacterial_pathway_table_20250523_081030.csv to mmrc-bacterial-pathway.

Uploaded Enzyme abundance table.csv as raw/Enzyme_abundance_table_20250523_081033.csv to mmrc-enzymes-abundance.

Uploaded NMR ASICS abundance table.csv as raw/NMR_ASICS_abundance_table_20250523_081034.csv to mmrc-nmr-asics-abundance.

Uploaded Metadata table.csv as raw/Metadata_table_20250523_081036.csv to mmrc-participant-metadata.

DynamoDB updated with new normalized IDs.

Successfully uploaded 99 unique records to S3

Figure 59 showing step 6: The UI confirms that all five CSVs have been uploaded successfully to their respective S3 buckets.

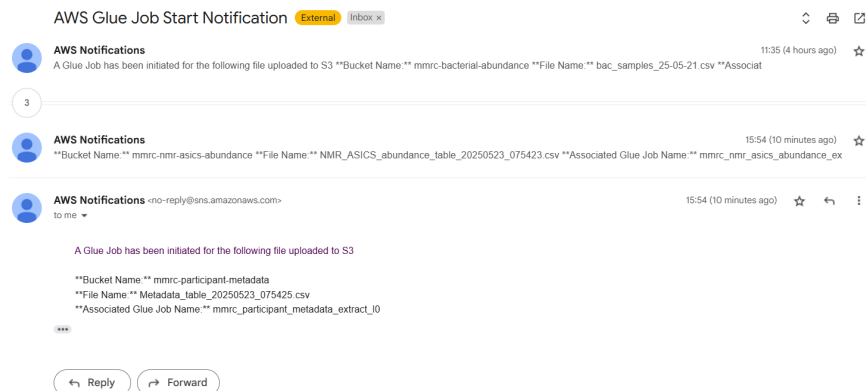


Figure 60 showing step 7: Multiple “AWS Glue Job Start Notification” arrives, indicating that S3 has received the files and backend processing is currently executing.

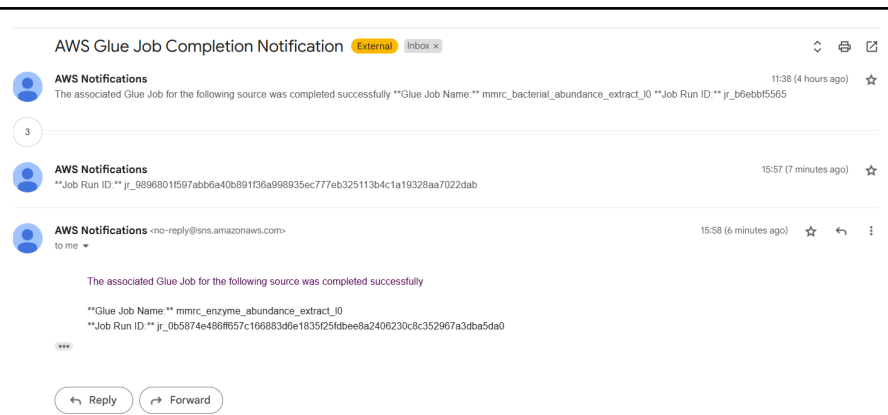


Figure 61 showing step 8: Once each Glue job is completed, a second SNS email—“AWS Glue Job Completion Notification”—confirms that the data have been added into the respective iceberg table in the staging subfolder.

☐	Name	Type	Last modified	Size	Storage class
☐	Bacterial_abundance_table_20250523_081029.csv	csv	May 23, 2025, 16:10:30 (UTC+08:00)	269.9 KB	Standard

☐	Name	Type	Last modified	Size	Storage class
☐	Bacterial_pathway_table_20250523_081030.csv	csv	May 23, 2025, 16:10:32 (UTC+08:00)	757.5 KB	Standard

☐	Name	Type	Last modified	Size	Storage class
☐	Enzyme_abundance_table_20250523_081033.csv	csv	May 23, 2025, 16:10:34 (UTC+08:00)	1.7 MB	Standard

☐	Name	Type	Last modified	Size	Storage class
☐	NMR_ASICS_abundance_table_20250523_081034.csv	csv	May 23, 2025, 16:10:36 (UTC+08:00)	125.1 KB	Standard

☐	Name	Type	Last modified	Size	Storage class
☐	Metadata_table_20250523_081036.csv	csv	May 23, 2025, 16:10:37 (UTC+08:00)	4.0 KB	Standard

Table: PK_Hash_Index - Items returned (99) 🔄 Actions ▾ Create item

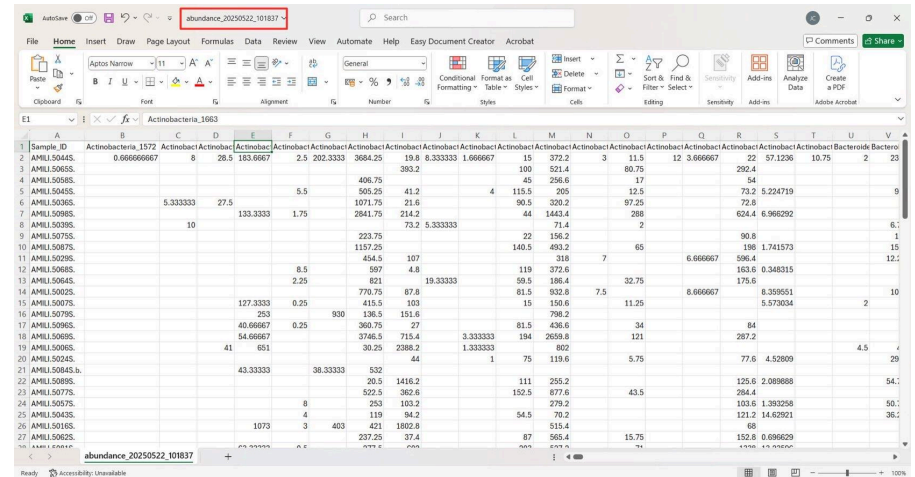
Scan started on May 23, 2025, 16:22:29

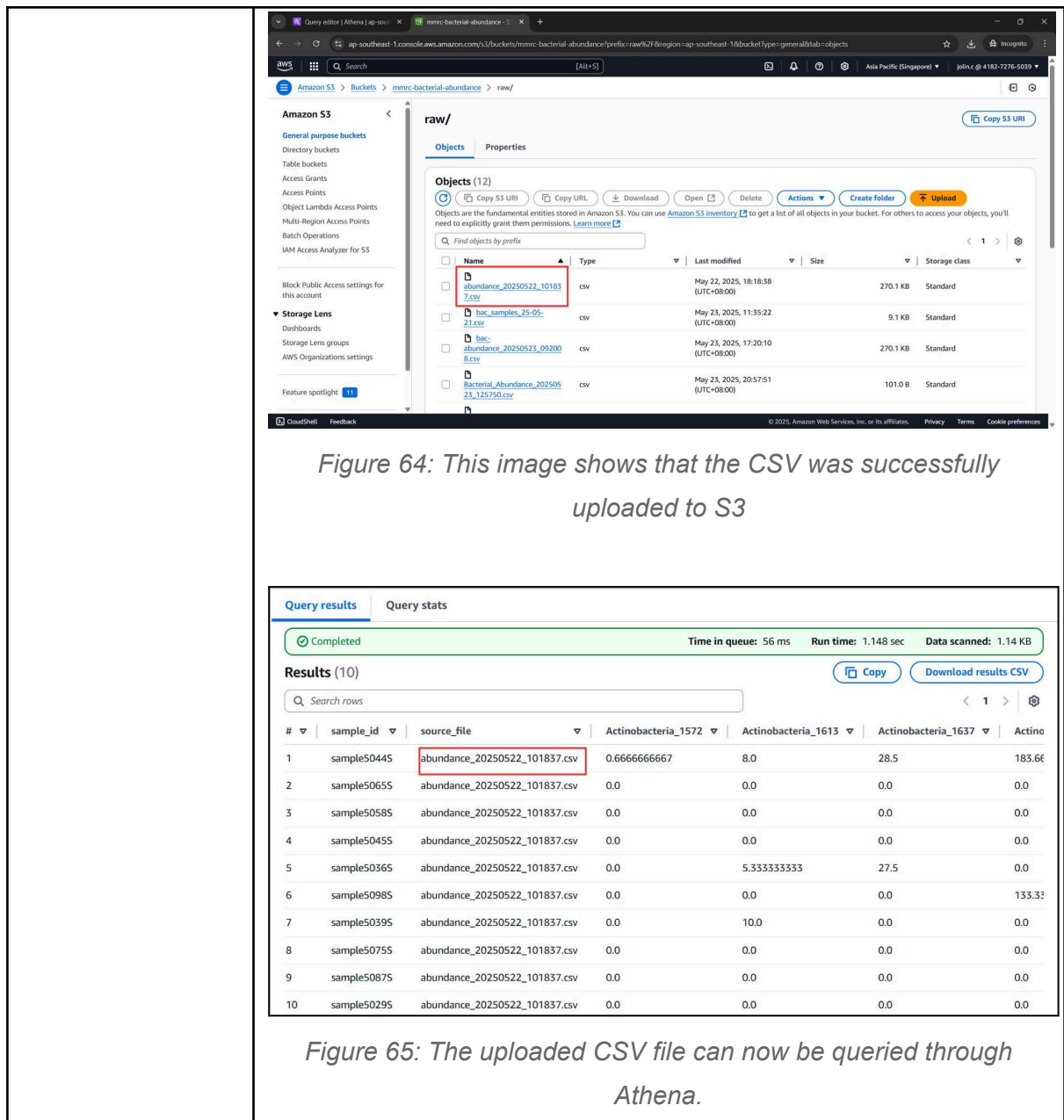
< 1 2 > ⚙️

☐	PK_Hash (String)
☐	6e686f3131a0c0d49bfa8cb9562164cf891f3e37253ddc781510464d8244760a
☐	33c22ac26695fd271e8106dc0a11e2711748fb940d1b8ae395a696266827770b
☐	491a89805507165fc44da607f6c021fdb5210da49f55236b61c124e27aa09573
☐	1210e6c2e909ec7ce4bf820e4de833f49af8070d34e39010fb552b95b3a0933f
☐	ef01091e6ef3295ab4a73f48efc578bed986806cb59e90c424493fad7d756c8d
☐	fd69b53f644e0e39b94d3e536daab93467b82d061cc31ca29755bf83f21cc98b
☐	ddd180a4466685b23ed9fee26d0d9cb5f081608808ccea9cf74a9ece83a5bf4
☐	0a239761346a9bb4d8568d50479cd3eac71aca71fd49df1910293b3d9a8d752a
☐	b6cfb9e281f64066b6cb04d5d9108276f4e3ecbc68c860831c481ba932591673

Figure 62 showing step 9: The AWS console can see each file under the raw subfolder in each bucket, and the PK_Hash_Index

	DynamoDB table is populated with the SHA-256 hashes of unique sample IDs.
--	---

Test ID	IT02
Component	Portal → S3 → Athena
What is being tested?	Verify uploaded data is queryable in Athena with correct schema tested?
How is it tested?	1. Upload valid CSV files for all 5 required tables via the portal. 2. Wait for S3 upload confirmation. 3. Query the table and compare against uploaded file (show that uploaded csv files have been added to staging)
Input	5 valid CSV files with known sample IDs and test values (e.g. bacterial-abundance.csv, etc.)
Expected Output	1. Files are present in the correct S3 raw/ path 2. Glue crawler loads them into the staging/ folder 3. Corresponding Athena tables are created/updated 4. Querying the tables in Athena returns matching rows from the uploaded CSVs
Actual Output	Matches expected exactly.
Result	Pass
Screenshot	 Figure 63: This image shows the CSV file and its contents before being uploaded to S3



Test ID	IT03
Component	Portal → DynamoDB → Athena
What is being tested?	Ensure IDs blocked by DynamoDB are excluded from Athena queries.
How is it tested?	1. Manually add a test ID (e.g., SAMPLE_9999) to DynamoDB PK_Hash table. 2. Upload a CSV containing SAMPLE_9999. 3. Verify the portal rejects SAMPLE_9999 during validation. 4. Query Athena to confirm SAMPLE_9999 is absent:
Input	Valid CSV file contains sample id (SAMPLE_9999)

Expected Output	
-----------------	---

Section 6: End User Testing

Test ID	EUT01
What is being tested?	Clarity of instructions and discoverability
How is it tested?	1. User opens the portal with no prior knowledge. 2. User scans the interface to understand what to do.
Expected Observation	1. Are the required files clearly labeled with "(Required)"? 2. Is there a tooltip or helper text explaining the workflow (e.g., "Upload all 5 files to begin validation")? 3. Does the UI highlight the next actionable step (e.g., "Upload files" buttons visually prominent)?
Actual Observation	1. Each of the five file-uploader widgets is clearly labeled with "(Required)" before its name. 2. A static helper message appears under the title: "Please upload all 5 required files." 3. No hover-tooltip is present, but the inline instruction text is sufficient. 4. The primary action button sits fixed at the bottom of the screen. On hover, both its label text and the adjacent flame icon turn bright red—providing a strong visual cue for the next step.
Output Evidence	

Data Upload Portal

Upload (Required) **bacterial-**
abundance.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

Upload (Required) **bacterial-**
pathway.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

Upload (Required) **enzymes-**
abundance.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

Upload (Required) **nmr-asics-**
abundance.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

Upload (Required) **participant-**
metadata.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

Upload (Optional) **new-taxonomy-**
IDs.csv

Drag and drop file here
Limit 1GB per file • CSV

Browse files

Figure 68: Every uploader slot is prefaced with the label “(Required)” before the file name.

Please upload all 5 required files.

Figure 69: Directly under the main header, a permanent prompt reads: “Please upload all 5 required files.”

Step 1: Remove Duplicates Within Files

No duplicates found within any files

Proceed to Step 2

Figure 70: a fixed action button anchors the bottom of the screen—on hover, both its text and the adjacent flame icon turn bright red.

Test ID	EUT02
What is being tested?	System responsiveness during file uploads.

How is it tested?	1. User uploads a large (50MB) CSV file for one of the required fields. 2. User waits for UI feedback.
Expected Observation	1. Is there a loading spinner/percentage indicator during upload? 2. Does the UI prevent duplicate uploads while processing (e.g., disable button)? 3. If upload fails (e.g., network error), is the error message actionable (e.g., "Retry" button)?
Actual Observation	1. As soon as a large CSV begins uploading, a horizontal progress bar appears beneath the file icon (blue fill moving left-to-right) instead of a spinner or percentage indicator. 2. If select or drop the same CSV again while it's still processing, the new upload immediately replaces (overwrites) the previous file and resets the progress bar. 3. When the upload fails, a red inline error message appears under the file icon and there is no "Retry" button or other actionable control—users must click the small "X" to remove the file and then re-select it to try again.

Output Evidence

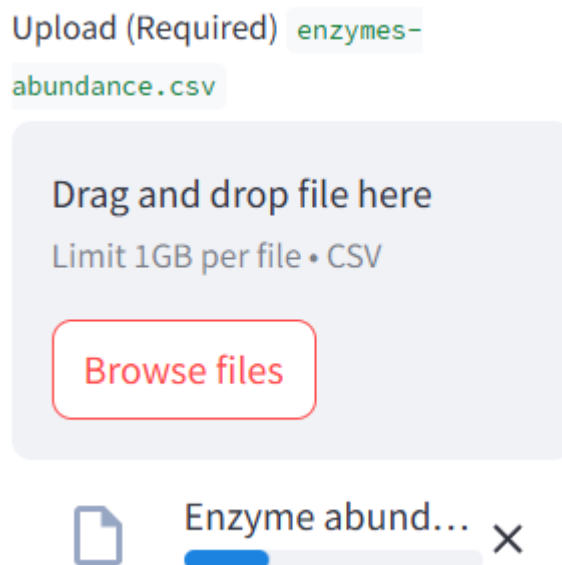
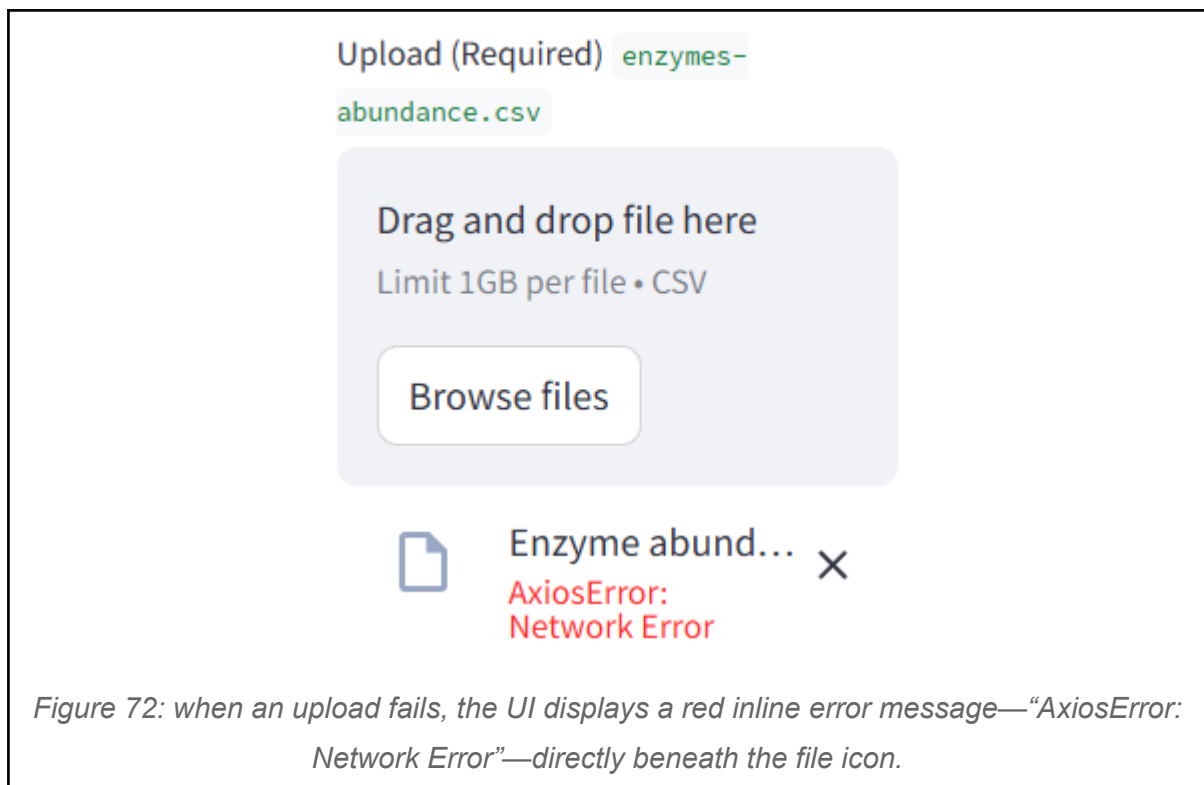


Figure 71: when a large CSV upload starts, a blue progress bar appears beneath its file icon—filling from left to right—instead of a spinner or percentage display.



Test ID	EUT03
What is being tested?	System’s response to invalid upload file type
How is it tested?	1. User tries to upload a file that is not of CSV format. User observes the system response.
Expected Observation	1. Is the error message specific (e.g., "Invalid format: Only CSV files accepted")? 2. Can the user easily remove/replace the file without restarting the workflow? 3. Does the UI retain other successfully uploaded files?
Actual Observation	1. Upon dropping a PDF, the UI displays a red inline error. 2. An “X” icon next to the filename lets the user remove the invalid file and immediately select a new one—no need to restart the entire workflow. 3. All previously uploaded, valid CSVs remain intact in their slots while the user removes the invalid file.
Output Evidence	

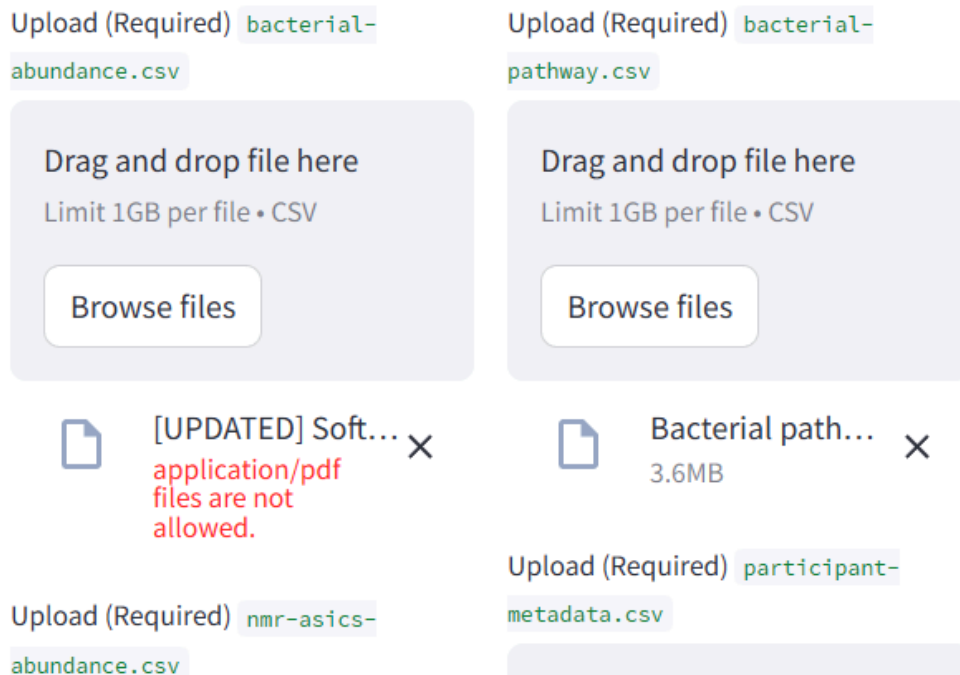


Figure 73: If a PDF is dropped, a red inline notification appears reading “application/pdf files are not allowed.” Also a small “X” button beside the filename lets the user instantly remove the invalid file and upload a correct one.

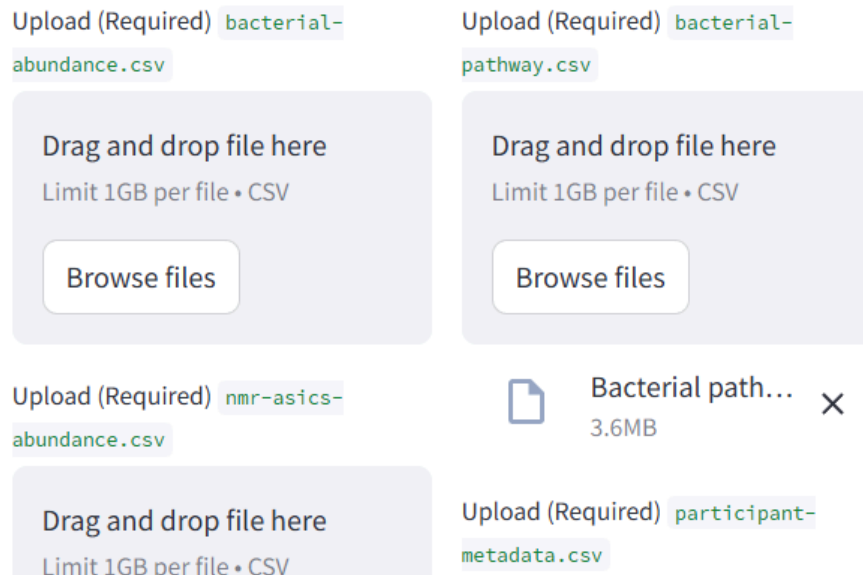


Figure 74: All valid CSVs already in place remain untouched while the user removes the bad file.

Test ID	EUT04
---------	-------

What is being tested?	Multi-tab session integrity
How is it tested?	1. User opens the portal in two browser tabs. 2. Uploads files in Tab A, then switches to Tab B.
Expected Observation	1. Tab B shows the original empty state (no file uploads from Tab A are visible). 2. System treats tabs as independent sessions (no synchronization). 3. No errors occur - each tab operates independently.
Actual Observation	1. When Tab A completes its uploads, Tab B remains on the initial empty upload screen—no files or progress carry over. 2. Actions in Tab A (uploads, error messages, buttons) do not appear in Tab B. 3. Both tabs function independently with no errors or cross-tab interference.

Output Evidence

The screenshot shows a web browser with two tabs, both labeled 'Streamlit'. The active tab displays the 'Data Upload Portal' at the URL 'mmr-dataake-data-upload.streamlit.app'. A red arrow points to the address bar. The portal features six upload slots arranged in a 2x3 grid. Each slot includes a 'Drag and drop file here' instruction, a 'Limit 1GB per file • CSV' note, and a 'Browse files' button. Below each button, a file icon and name are shown, along with a close button (X). The files are: 'Bacterial abund...' (1.2MB), 'Bacterial path...' (3.6MB), 'Enzyme abund...' (8.3MB), 'NMR ASICS abu...' (108.5KB), 'Metadata table...' (9.6KB), and 'new-taxonomy-IDs.csv' (Optional).

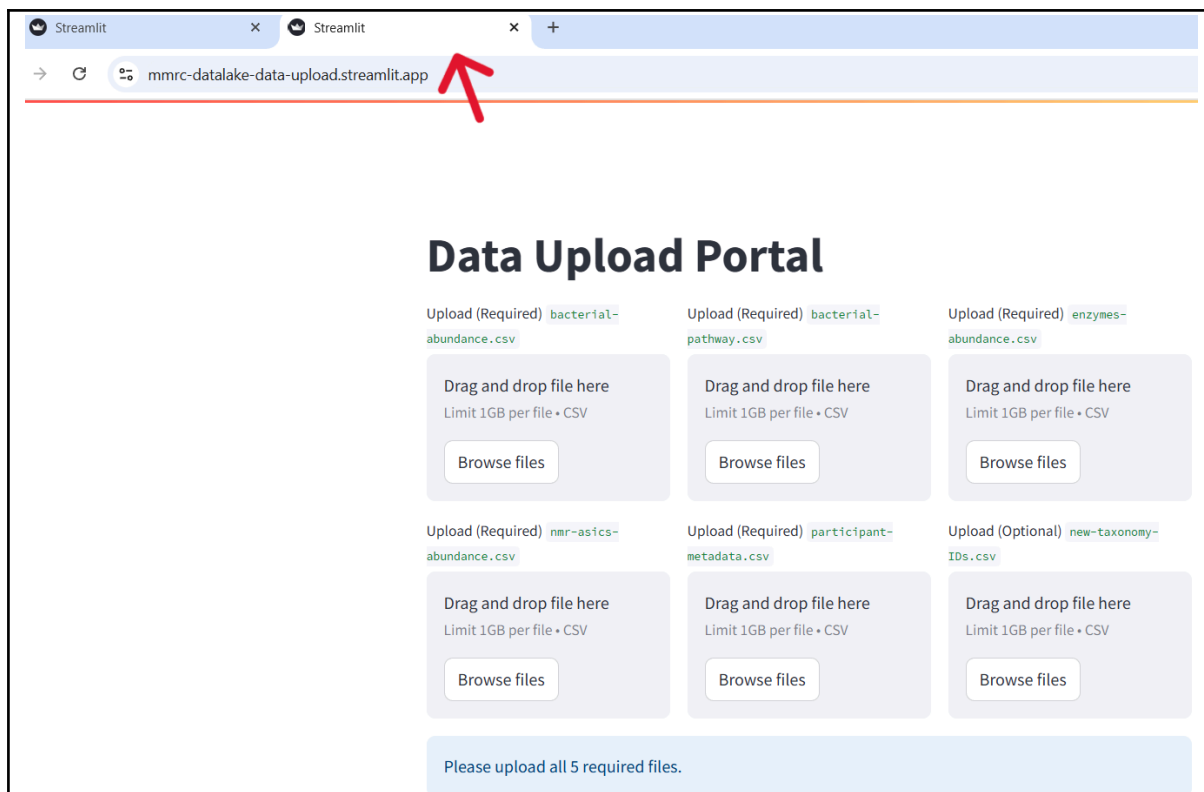
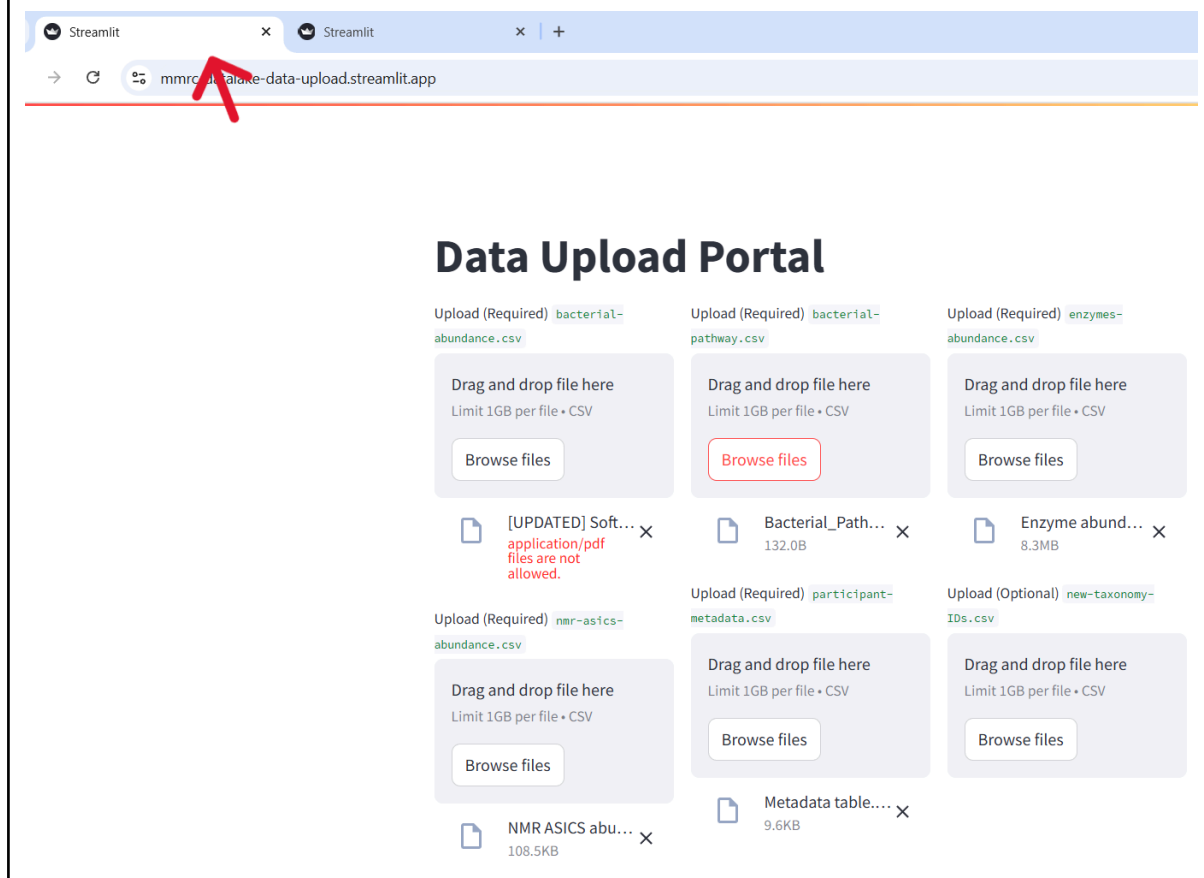
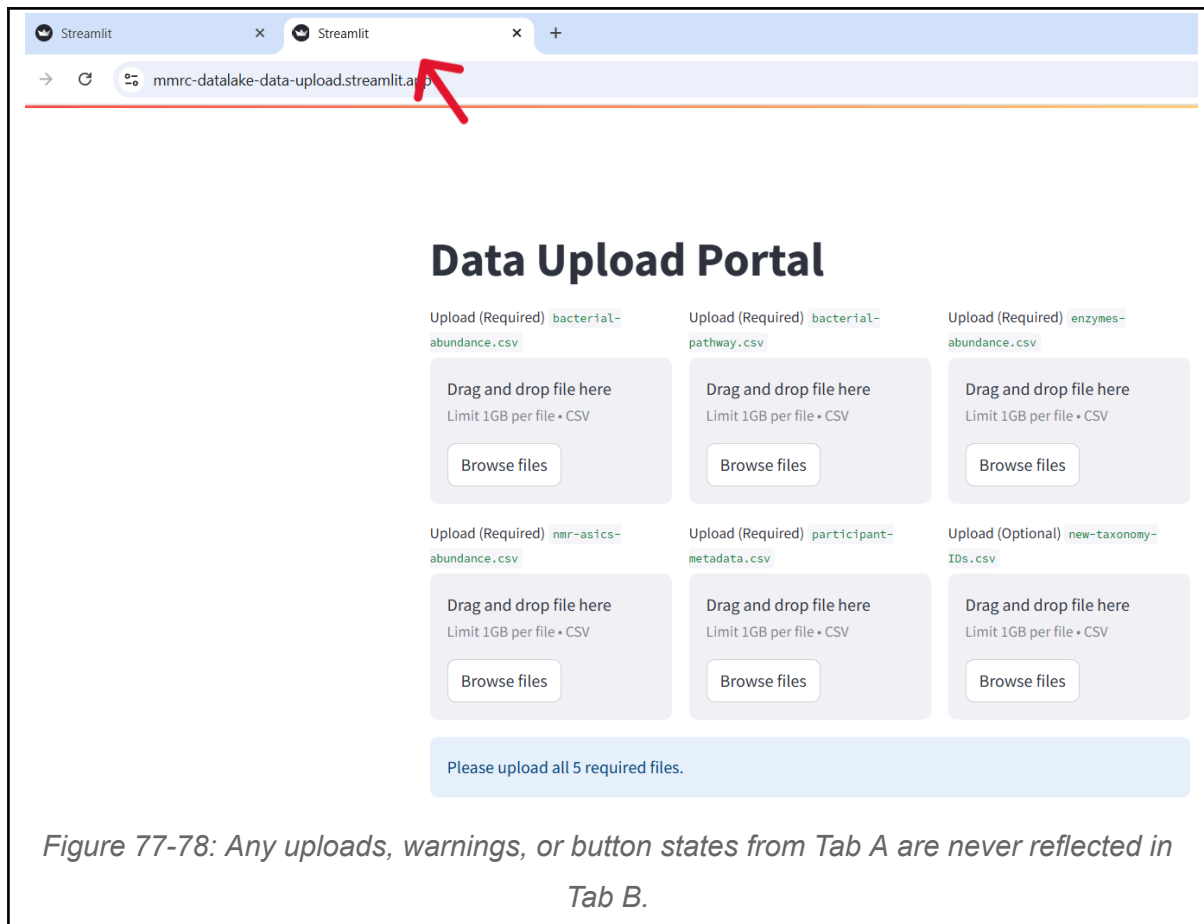


Figure 75-76: After Tab A finishes uploading, Tab B still shows the empty upload screen—none of Tab A's files or progress appear.





Test ID	EUT05
What is being tested?	Mobile Responsiveness
How is it tested?	1. User accesses the portal on a smartphone. 2. Attempts to upload files and navigate steps.
Expected Observation	1. Are file upload buttons and tables readable without zooming? 2. Is touch-target size adequate (e.g., buttons >48px)?
Actual Observation	1. Layout & Readability: All file-upload fields and labels automatically resize to the screen width; text remains legible without pinch-zoom. 2. Touch Targets: The primary buttons (“Browse files”, “Proceed to Step 2”) are approximately 48 px tall, meeting touch-target guidelines. However, the small “X” file-remove icon is only ~24 px and can be tricky to tap accurately on a phone.
Output Evidence	

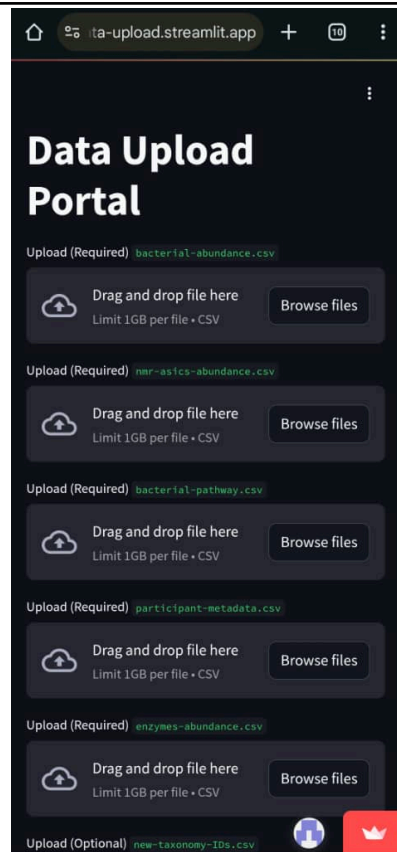
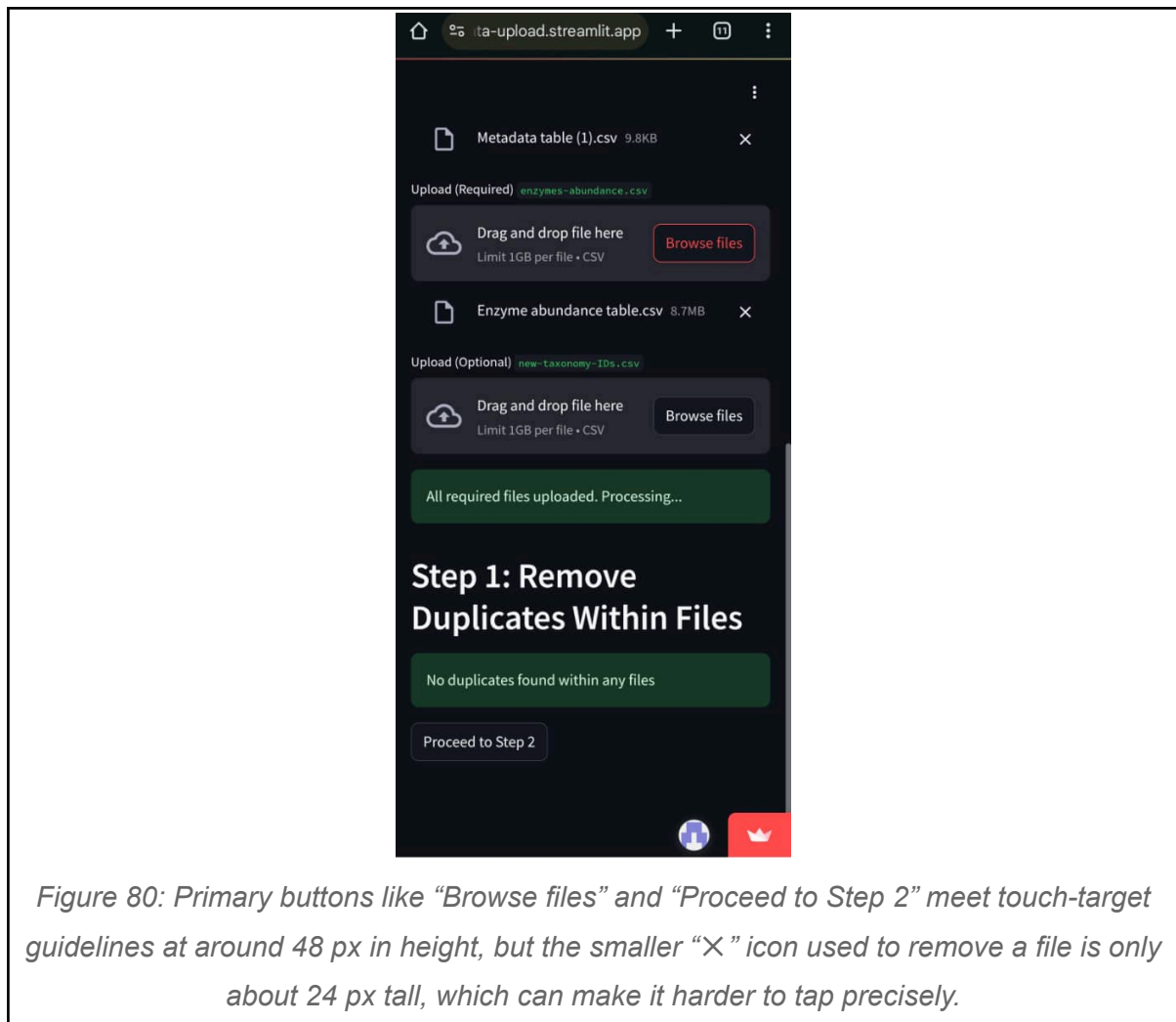


Figure 79: On mobile, the upload fields and their labels automatically scale to fit the screen width, keeping all text crisp and readable without any pinch-to-zoom.



Test ID	EUT06
What is being tested?	Cross-browser compatibility
How is it tested?	1. Test portal on Chrome, Edge, and Safari. 2. Verify all UI elements and workflows function.
Expected Observation	1. Consistent rendering and functionality across browsers. 2. No JavaScript errors in the console.
Actual Observation	1. Chrome, Edge: UI elements render consistently; drag-and-drop, validation steps, and upload workflows all function without issues. No JavaScript errors in the developer console. 2. Safari (macOS & iOS): Layout and functionality are intact, but the bottom action button slightly overlaps the mobile

Safari safe-area inset on iOS—requiring a quick vertical scroll to access. No console errors reported.

Output Evidence

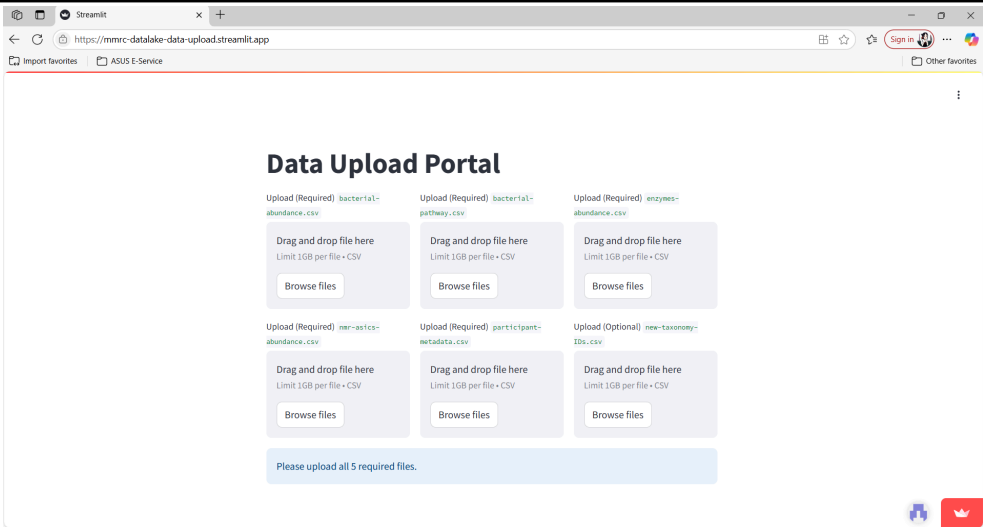


Figure 81: Shows the portal running as expected on Edge.

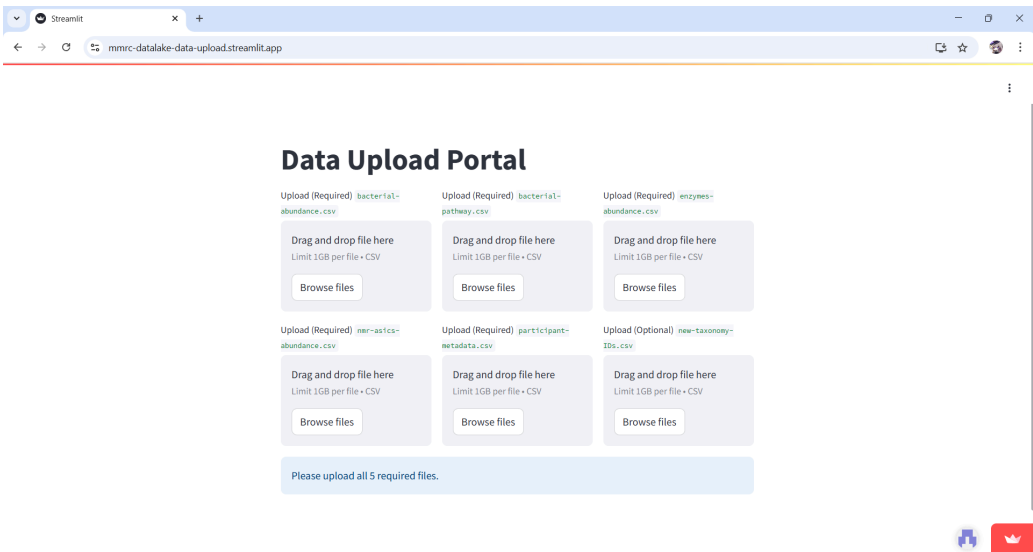
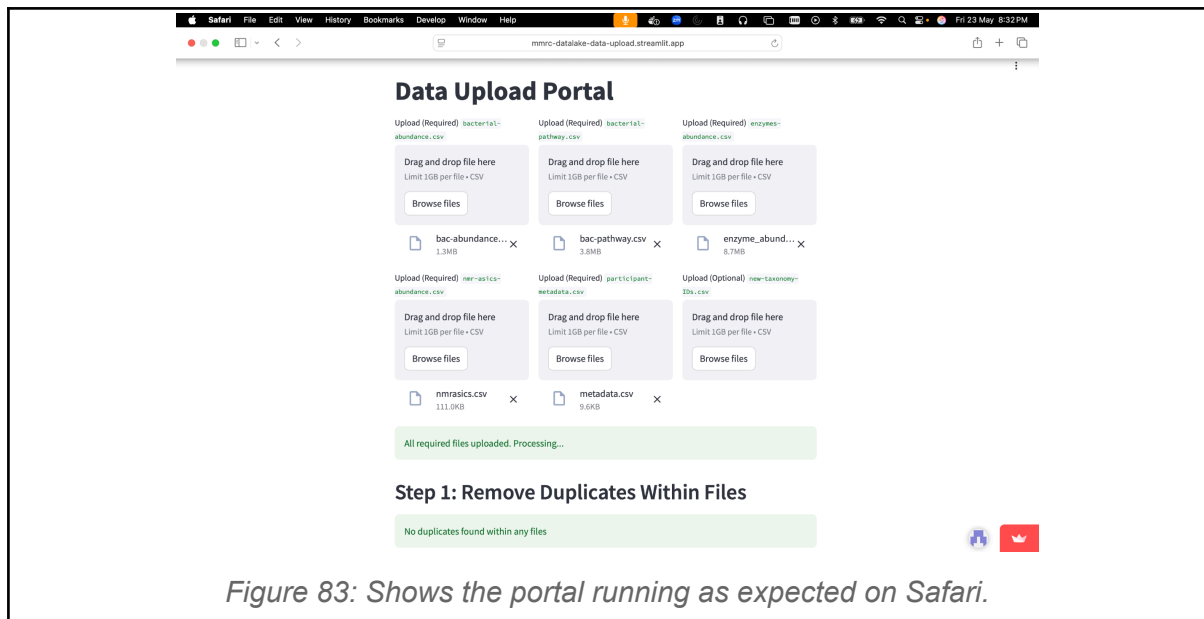


Figure 82: Shows the portal running as expected on Chrome.



Test ID	EUT07
What is being tested?	Post-upload clarity and confirmation
How is it tested?	1. User completes a successful upload. 2. Look for confirmation or follow-up actions.
Expected Observation	1. Is there a clear success message with a summary (e.g., "5 files uploaded to S3")? 2. Can they start a new upload immediately without manual refresh?
Actual Observation	1. The system displays a detailed success message after uploads, including: a. Individual confirmation for each file (e.g., "Uploaded 'bacterial-abundance.csv' as 'raw/bacterial-abundance_20240523.csv' to 'mmrc-bacterial-abundance'") b. A DynamoDB update confirmation (e.g., "DynamoDB updated with new normalised IDs") c. A summary line with the total records uploaded (e.g., "Successfully uploaded 25 unique records to S3") 2. The portal automatically resets after a successful upload:

- a. The file upload fields are cleared.
- b. Session state is refreshed (no residual data from the previous upload).
- c. Users can immediately drag/drop new files without needing to reload the page.

Output Evidence

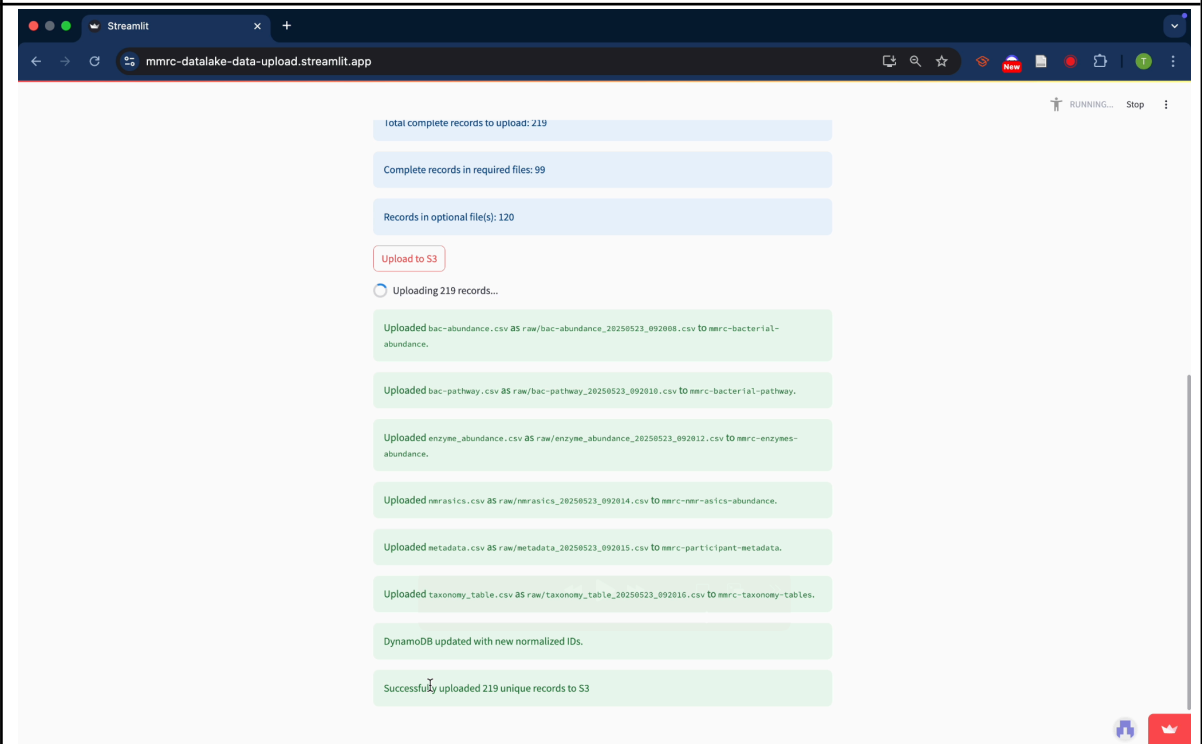


Figure 84: shows success message upon the files are uploaded to their respective S3 buckets

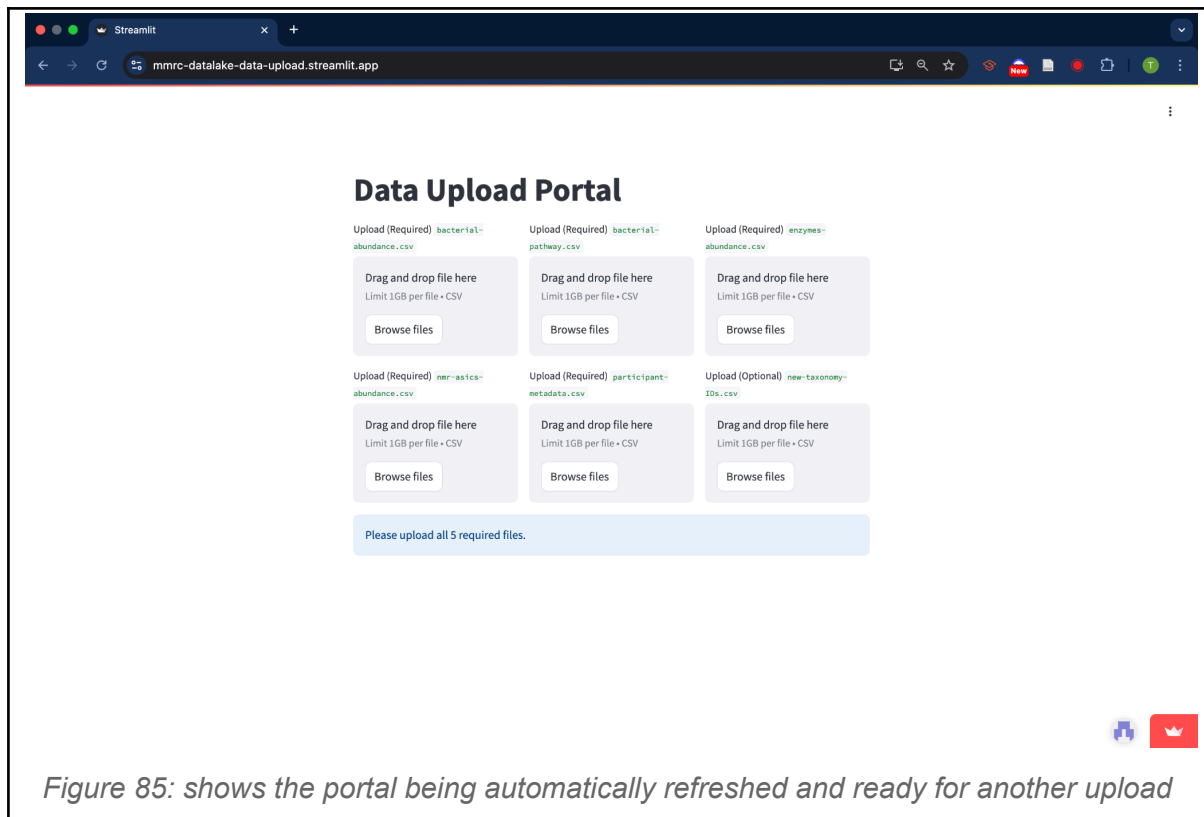


Figure 85: shows the portal being automatically refreshed and ready for another upload

Test ID	EUT08
What is being tested?	S3 access restriction to raw/ folder only
How is it tested?	1. Log in to the AWS Console as the IAM test user with S3AthenaAccess policy attached. 2. Navigate to the S3 console. 3. Select an assigned bucket (e.g., mmrc-bacterial-abundance).
Expected Observation	1. User can list, view, and upload to raw/ folder 2. User cannot open, or upload to staging/ folder 3. Access denied messages appear if the user tries to enter unauthorized paths
Actual Observation	1. Access given when attempting to enter raw/ folder 2. Access denied when attempting to enter staging/ folder (AccessDenied: s3:ListBucket and s3:GetObject errors triggered)
Output Evidence	

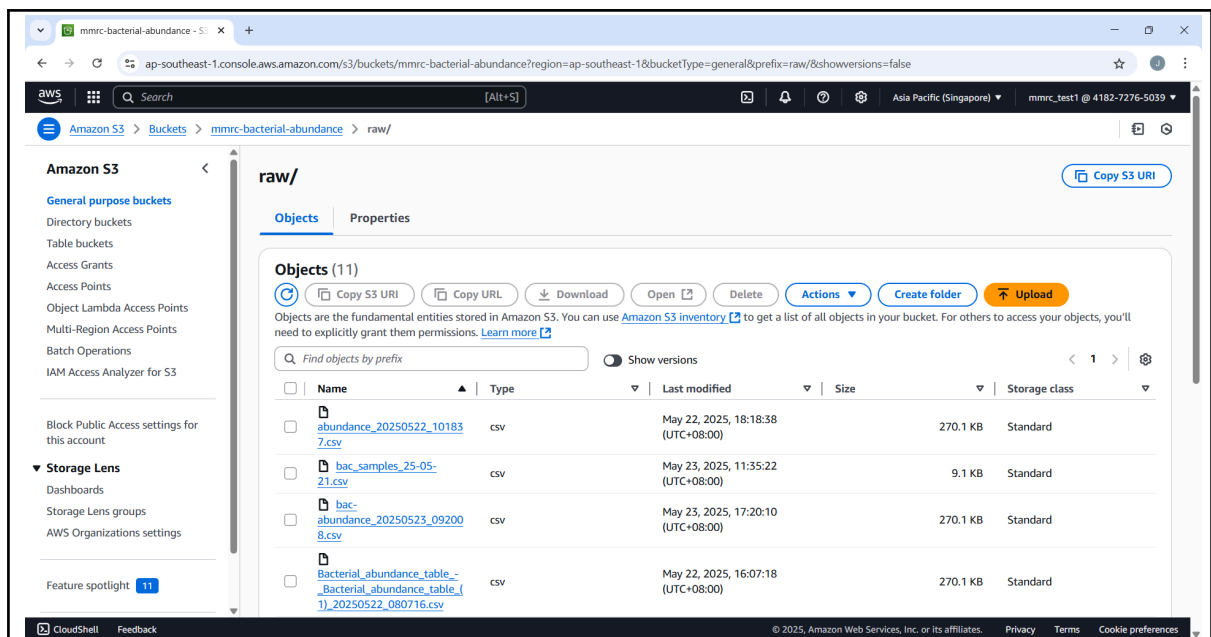


Figure 86: The operation-level account has full access to the raw subfolder.

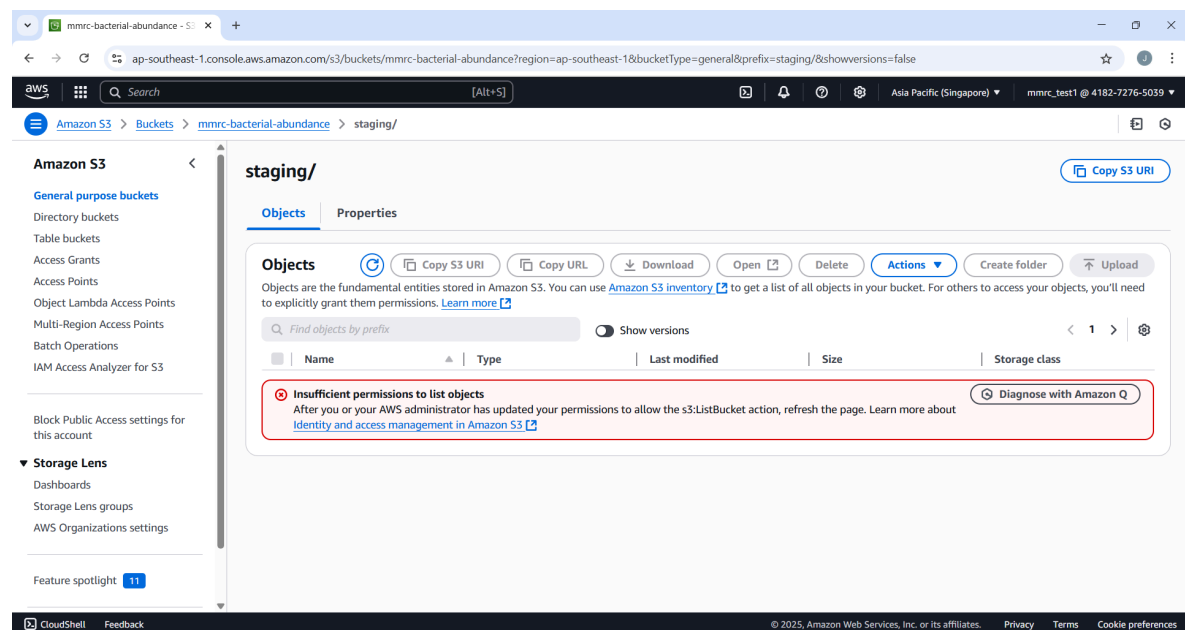


Figure 87: The operation-level account has no access to the staging subfolder.

Test ID	EUT09
What is being tested?	Athena table preview functionality
How is it tested?	1. User navigates to the Athena console in AWS.

	2. Expands "Tables" dropdown and clicks the 3-dot menu next to a table (e.g., mmrc_bacterial_abundance_sam). 3. Select "Preview table" from the menu.
Expected Observation	1. Preview query (SELECT * FROM "mmrc_bacterial_abundance_sam" LIMIT 10) executes automatically. 2. Results load within ≤30 seconds for first time load (for tables ≤1GB). 3. Columns and sample data display in a readable tabular format.
Actual Observation	1. Preview query initiated with one click. 2. Results displayed in ~10 seconds for a first time load (minor UI lag during execution). 3. Output matched table schema expectations.

Output Evidence

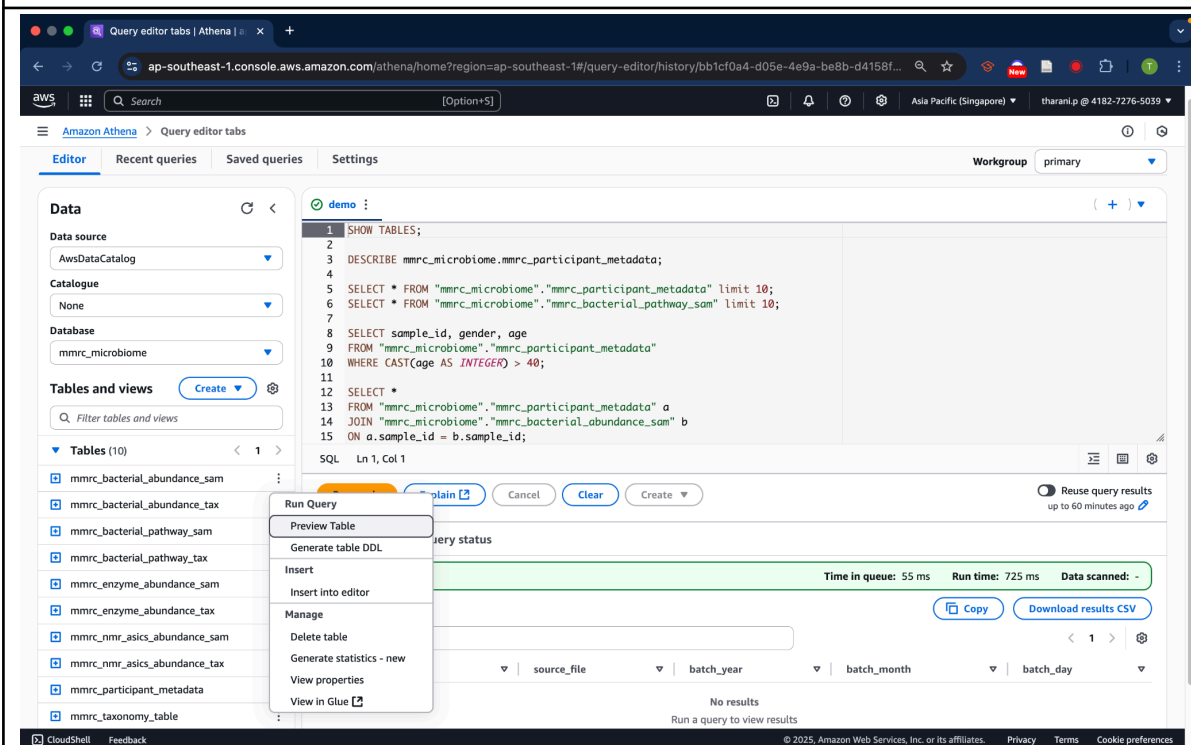


Figure 88: User initiates table preview by clicking the 3-dot menu and selecting 'Preview table' in the Athena console.

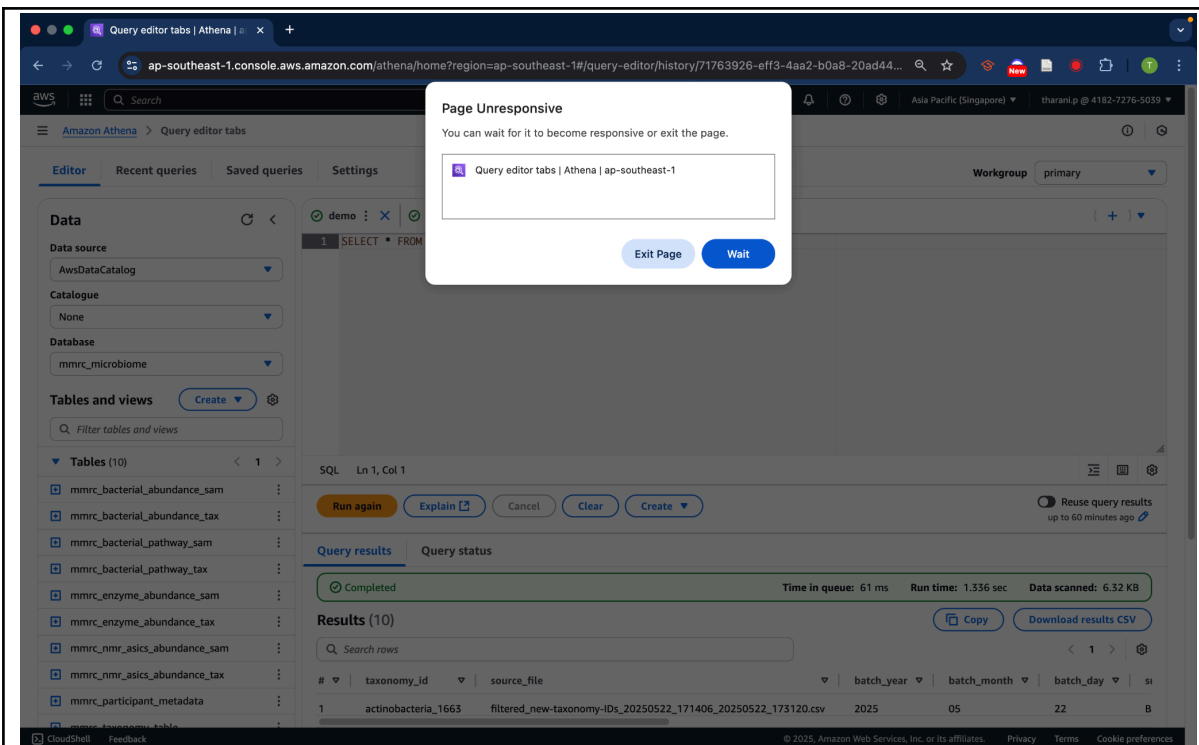


Figure 89: Browser displays a 'Leave site?' warning during query execution, indicating UI unresponsiveness for ~3 seconds. Users might mistakenly interpret this as a system failure.

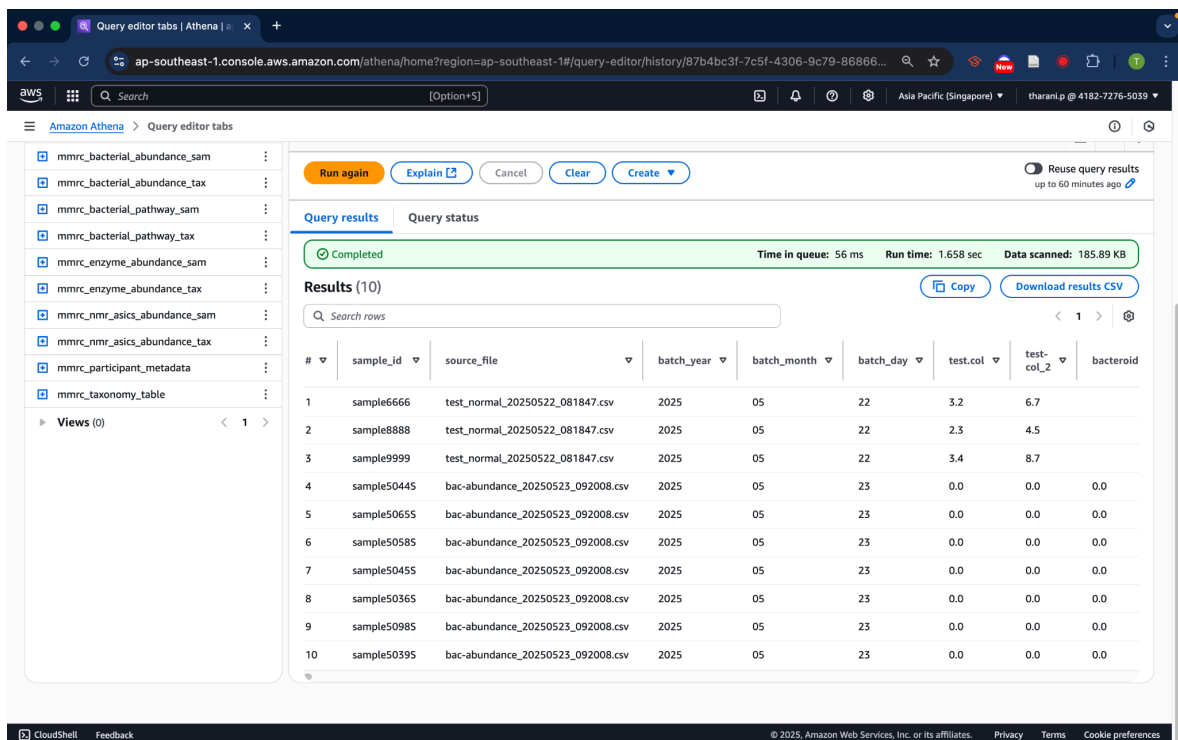


Figure 90: Preview results (10 rows) load within 10 seconds for a first time load, displaying correctly formatted columns and sample data. The output matches the expected table schema.

Section 7: Recommendations for Improvement

7.1 Enhancement to front-end validation for CSV files

During development of the front-end interface, only mock data and sample datasets provided by the research team were used for testing. All files had complete column names and all cell values were filled in with appropriate data values. However, during the software demonstration component of our final presentation, the software had allowed multiple corrupted instances of csv files to be uploaded to S3. As such, the data transformation pipeline on AWS had failed as it could not read the corrupted data. No data instances were populated into our Iceberg tables that can be queried on Athena.

As such, the pre-upload validation has to be enhanced to thoroughly validate the following:

- Checking for inconsistent row lengths, encoding errors or unparsable content potentially indicating a corrupted file.
- Validate structure of file received, ensuring all column header values are present
- Checking for incorrect column data types and null values

7.2 Optimization of processing speed for the web interface.

Testing for the front-end component was mostly done with a local instance of our Streamlit app running without much delay in processing speed. However, once it was deployed using Streamlit's community cloud as a web app, we noticed significant delays in the processing time during each of the validation steps. This is most likely due to network communication overhead and also lack of parallelism in our code. Our code should be refactored to leverage on parallel computing practices using the multiprocessing library in Python to parallelise the validation steps for each of the upload files to significantly decrease processing time. Additionally, the deployment strategy could be enhanced to package our front-end codebase in a Docker container that can be hosted on an AWS EC2 on-demand server instance for more granular deployment environment controls.

7.3 Improve the clarity of error messages

AWS Simple Notification Service (SNS) has been configured to send email notification to registered users to notify any changes in state of a Glue job's execution. However, the error message shown during the failure of a Glue job can be ambiguous for non-technical users who have no domain knowledge on the inner-workings of the data transformation pipeline. Improvements can be made to identify and map some commonly-occurring code error messages that arise from user-error with predefined reworded explanations free from any technical jargon to increase users' understanding.

Section 8: Test Limitations

8.1 Limited testing with diverse, dynamically structured real-world datasets.

As mentioned previously, testing for all components for the front-end and data lake were either using sample datasets or mock data which we had tried to replicate its structure to closely resemble real instances. However, as we began to be exposed to the kinds of experiments carried out by the researchers, we understood that microbiome datasets are very dynamic as hundreds of new rows and columns with varied data can be synthesized from new experiments. As such, if the researchers were to expand the functionality of the data lake for new research projects, the current iteration of the solution would not be able to transform any stored data to database tables for queries. A new team of developers would have to be brought in to iterate upon the current solution.

8.2 Current error messages may not be sufficiently clear

During end-user testing, researchers had frequently noted that they were not able to fully understand the error messages delivered in a Glue Job failure email notification. As such, they were unable to deduce their mistakes and recover from this error. Due to the limited testing time and face to face meeting opportunities, we were unable to sit down with the researchers for extended periods of time to observe the most common error messages received. With extended testing, we can include a portion in our user guide manual detailing commonly received error messages and their potential causes.

8.3 Unperformed Tests

Below is a table detailing some of the planned tests that the team was unable to complete during this development phase and their potential impact on the software's performance.

Tests NOT Conducted	Reason Not Tested	Potential Impact	How To Test
Streamlit Timeout Handling	Assumed typical file uploads would complete within seconds/minutes. Tested only with small files (<10MB), not large (~1GB) or slow-network scenarios.	Partial Uploads: Large files may fail mid-transfer, leaving incomplete/corrupt data in S3. User Confusion: No timeout warnings could lead to frozen UI or lost progress.	Simulate Timeout: Forcefully kill the Streamlit process after 3 minutes during upload. Large File Test: Generate a 1GB test CSV and monitor upload progress. Expected Behavior: 1. Clear timeout error message 2. Automatic cleanup of partial S3 uploads.
Concurrent User Uploads	Assumed single-user usage pattern.	Race Conditions: DynamoDB duplicate checks could miss collisions if two users upload the same ID simultaneously. S3 Corruption: Parallel writes to the same S3 key may overwrite data.	Use Locust to simulate 3+ users uploading files simultaneously:
Malformed CSV Edge Cases	Focused on well-formatted CSVs	Silent Corruption: Non-UTF-8 encodings or binary files may parse incorrectly. Crashes: Missing headers or broken delimiters could break processing.	Upload files with: 1. UTF-16 encoding. 2. Tab-delimited (non-CSV) data. 3. Missing headers. Expected Behavior: Rejection with descriptive errors (e.g., "Invalid CSV: UTF-16 not supported").